Contents

1 All Macros **1**

2 Data Structure **1**
- 2.1 DSU With Rollbacks 1
- 2.2 Binary Trie 2
- 2.3 BIT-2D 2
- 2.4 MO with Update 2

3 Geometry **2**
- 3.1 Line 2
- 3.2 Point 4
- 3.3 circle 5
- 3.4 polygon 7

4 Graph **11**
- 4.1 Lifting, LCA, HLD 11
- 4.2 Virtual Tree 12
- 4.3 Bridge Tree 12

5 Mahir **13**
- 5.1 temp 13

6 Math **20**
- 6.1 Linear Sieve 20
- 6.2 Pollard Rho 20
- 6.3 Chinese Remainder Theorem 21
- 6.4 Mobius Function 21

7 String **21**
- 7.1 Double hash 21
- 7.2 Suffix Array 22
- 7.3 Z Algo 22

Sublime Build

```
{ "cmd" : ["g++ -std=c++20 -DLOCAL -Wall $file_name -o
    $file_base_name && timeout 5s ./$file_base_name
    output.txt && rm $file_base_name"], "selector" : "
    source.cpp", "file_regex": " (..[ :]*):([0-9]+)
    :?([0-9]+)?:? (.*)$", "shell": true, "working_dir" :
     "$file_path" }

 { "shell_cmd": "g++ -std=c++23 \"$file\" -o \"
     $file_base_name.exe\" && \"$file_base_name.exe\" <
     input.txt > output.txt && del \"$file_base_name.exe
     \"", "working_dir": "$file_path", "selector": "
     source.c++" }

// interactive "shell_cmd": "g++ -std=c++23 \"$file\" -o
    \" $file_base_name.exe\" && start cmd /k \"
    $file_base_name. exe\"",
```

1 All Macros

```
// #pragma GCC optimize("Ofast")
// #pragma GCC optimization ("O3")
// #pragma comment(linker, "/stack:200000000")
// #pragma GCC optimize("unroll-loops")
// #pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,
    abm,mmx,avx,tune=native")
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
typedef long long ll;
#define ld long double
#define ull unsigned long long
#define F first
#define S second
#define gcd __gcd
#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define pb push_back
#define eb emplace_back
#define lb lower_bound
#define ub upper_bound
using namespace std;
using namespace __gnu_pbds;

//typedef tree<int,null_type,less<int>,rb_tree_tag,
    tree_order_statistics_node_update> ordered_set;
//typedef tree<pair<ll, ll>, null_type, less<pair<ll, ll
    >>, rb_tree_tag, tree_order_statistics_node_update>
    ordered_multiset;

struct custom_hash {
 static uint64_t splitmix64(uint64_t x) {
  // http://xorshift.di.unimi.it/splitmix64.c
  x += 0x9e3779b97f4a7c15;
  x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
  x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
  return x ^ (x >> 31);
 }

 size_t operator()(uint64_t x) const {
  static const uint64_t FIXED_RANDOM = chrono::
      steady_clock::now().time_since_epoch().count();
  return splitmix64(x + FIXED_RANDOM);
 }
};

constexpr ll inf = 1e18;
const ll N = 1000000;
const ll M = 505;
const ll MOD = 1e9 + 7; // 998244353 // 1000000007

void solve() {

}

int main()
{
 ios_base::sync_with_stdio(false);
 cin.tie(NULL);

 pre();

 int tc = 1, C = 1;
 //cin >> tc;
 cout << fixed << setprecision(10);
 while (tc--) {
  //cout << "Case " << C++ << ": ";

  solve();
 }
}
```

2 Data Structure

2.1 DSU With Rollbacks

```
struct Rollback_DSU {
 int n;
 vector<int> par, sz;
 vector<pair<int, int>> op;
 Rollback_DSU(int n) : par(n), sz(n, 1) {
  iota(par.begin(), par.end(), 0);
  op.reserve(n);
 }
 int Anc(int node) {
  for (; node != par[node]; node = par[node])
   ; // no path compression
  return node;
 }
 void Unite(int x, int y) {
  if (sz[x = Anc(x)] < sz[y = Anc(y)]) swap(x, y);
  op.emplace_back(x, y);
  par[y] = x;
  sz[x] += sz[y];
 }
 void Undo(int t) {
  for (; op.size() > t; op.pop_back()) {
   par[op.back().second] = op.back().second;
   sz[op.back().first] -= sz[op.back().second];
  }
 }
};

// alternate
vector<int> par, height;

void init (int n) {
  height.assign(n + 1, 0);
  par.resize(n + 1);
  for (int i = 1; i <= n; ++i)
    par[i] = i;
}

int find (int node) {
  if (par[node] != node) {
    par[node] = find(par[node]);
  }
  return par[node];
}

void unite (int A, int B) {
  if (A == B)
    return;
  int rootA = find(A);
  int rootB = find(B);
  if (height[rootA] <= height[rootB])
    swap(rootA, rootB);
  par[rootB] = rootA;
  if (height[rootA] == height[rootB])
    height[rootA]++;
}
```

2.2 Binary Trie

```
const int N = 1e7 + 5, b = 30;
int tc = 1;
struct node {
 int vis = 0;
 int to[2] = {0, 0};
 int val[2] = {0, 0};
 void update() {
  to[0] = to[1] = 0;
  val[0] = val[1] = 0;
  vis = tc;
 }
} T[N + 2];
node *root = T;
int ptr = 0;
node *nxt(node *cur, int x) {
 if (cur->to[x] == 0) cur->to[x] = ++ptr;
 assert(ptr < N);
 int idx = cur->to[x];
 if (T[idx].vis < tc) T[idx].update();
 return T + idx;
}
int query(int j, int aj) {
 int ans = 0, jaj = j ^ aj;
 node *cur = root;
 for (int k = b - 1; ~k; k--) {
  maximize(ans, nxt(cur, (jaj >> k & 1) ^ 1)->val[1 ^ (aj
       >> k & 1)]);
  cur = nxt(cur, (jaj >> k & 1));
 }
 return ans;
}
void insert(int j, int aj, int val) {
 int jaj = j ^ aj;
 node *cur = root;
 for (int k = b - 1; ~k; k--) {
  cur = nxt(cur, (jaj >> k & 1));
  maximize(cur->val[j >> k & 1], val);
 }
}
void clear() {
 tc++;
 ptr = 0;
 root->update();
}
```

2.3 BIT-2D

```
const int N = 1008;
int bit[N][N], n, m;
int a[N][N], q;
void update(int x, int y, int val) {
 for (; x < N; x += -x & x)
  for (int j = y; j < N; j += -j & j) bit[x][j] += val;
}
int get(int x, int y) {
 int ans = 0;
 for (; x; x -= x & -x)
  for (int j = y; j; j -= j & -j) ans += bit[x][j];
 return ans;
}
int get(int x1, int y1, int x2, int y2) {
 return get(x2, y2) - get(x1 - 1, y2) - get(x2, y1 - 1) +
      get(x1 - 1, y1 - 1);
}
```

2.4 MO with Update

```
const int N = 1e5 + 5, sz = 2700, bs = 25;
int arr[N], freq[2 * N], cnt[2 * N], id[N], ans[N];
struct query {
  int l, r, t, L, R;
  query(int l = 1, int r = 0, int t = 1, int id = -1)
      : l(l), r(r), t(t), L(l / sz), R(r / sz) {}
  bool operator<(const query &rhs) const {
    return (L < rhs.L) or (L == rhs.L and R < rhs.R) or
          (L == rhs.L and R == rhs.R and t < rhs.t);
  }
} Q[N];
struct update {
  int idx, val, last;
} Up[N];
int qi = 0, ui = 0;
int l = 1, r = 0, t = 0;

void add(int idx) {
  --cnt[freq[arr[idx]]];
  freq[arr[idx]]++;
  cnt[freq[arr[idx]]]++;
}
void remove(int idx) {
  --cnt[freq[arr[idx]]];
  freq[arr[idx]]--;
  cnt[freq[arr[idx]]]++;
}
void apply(int t) {
  const bool f = l <= Up[t].idx and Up[t].idx <= r;
  if (f) remove(Up[t].idx);
  arr[Up[t].idx] = Up[t].val;
  if (f) add(Up[t].idx);
}
void undo(int t) {
  const bool f = l <= Up[t].idx and Up[t].idx <= r;
  if (f) remove(Up[t].idx);
  arr[Up[t].idx] = Up[t].last;
  if (f) add(Up[t].idx);
}
int mex() {
  for (int i = 1; i <= N; i++)
    if (!cnt[i]) return i;
  assert(0);
}
int main() {
  sort(id + 1, id + qi + 1, [&](int x, int y) { return Q[
       x] < Q[y]; });
  for (int i = 1; i <= qi; i++) {
    int x = id[i];
    while (Q[x].t > t) apply(++t);
    while (Q[x].t < t) undo(t--);
    while (Q[x].l < l) add(--l);
    while (Q[x].r > r) add(++r);
    while (Q[x].l > l) remove(l++);
    while (Q[x].r < r) remove(r--);
    ans[x] = mex();
  }
}
```

3 Geometry

3.1 Line

```
struct line {
 PT a, b; // goes through points a and b
 PT v; double c; //line form: direction vec [cross] (x, y
     ) = c
 line() {}
 //direction vector v and offset c
 line(PT v, double c) : v(v), c(c) {
  auto p = get_points();
  a = p.first; b = p.second;
 }
 // equation ax + by + c = 0
 line(double _a, double _b, double _c) : v({_b, -_a}), c
     (-_c) {
  auto p = get_points();
  a = p.first; b = p.second;
 }
 // goes through points p and q
 line(PT p, PT q) : v(q - p), c(cross(v, p)), a(p), b(q)
     {}
  pair<PT, PT> get_points() { //extract any two points
       from this line
```

```
 PT p, q; double a = -v.y, b = v.x; // ax + by = c
 if (sign(a) == 0) {
  p = PT(0, c / b);
  q = PT(1, c / b);
 }
 else if (sign(b) == 0) {
  p = PT(c / a, 0);
  q = PT(c / a, 1);
 }
 else {
  p = PT(0, c / b);
  q = PT(1, (c - a) / b);
 }
 return {p, q};
 }
// ax + by + c = 0
array<double, 3> get_abc() {
 double a = -v.y, b = v.x;
 return {a, b, -c};
}
// 1 if on the left, -1 if on the right, 0 if on the
    line
int side(PT p) { return sign(cross(v, p) - c); }
// line that is perpendicular to this and goes through
    point p
line perpendicular_through(PT p) { return {p, p + perp(v
    )}; }
// translate the line by vector t i.e. shifting it by
    vector t
line translate(PT t) { return {v, c + cross(v, t)}; }
// compare two points by their orthogonal projection on
    this line
// a projection point comes before another if it comes
    first according to vector v
bool cmp_by_projection(PT p, PT q) { return dot(v, p) <
    dot(v, q); }
line shift_left(double d) {
 PT z = v.perp().truncate(d);
 return line(a + z, b + z);
}
double dist(PT p) {
 return fabs(cross(v, p) - c) / v.norm();
}
};
// find a point from a through b with distance d
PT point_along_line(PT a, PT b, double d) {
 assert(a != b);
 return a + (((b - a) / (b - a).norm()) * d);
}
```

```
// projection point c onto line through a and b assuming
    a != b
PT project_from_point_to_line(PT a, PT b, PT c) {
 return a + (b - a) * dot(c - a, b - a) / (b - a).norm2()
     ;
}
// reflection point c onto line through a and b assuming
    a != b
PT reflection_from_point_to_line(PT a, PT b, PT c) {
 PT p = project_from_point_to_line(a,b,c);
 return p + p - c;
}
// minimum distance from point c to line through a and b
double dist_from_point_to_line(PT a, PT b, PT c) {
 return fabs(cross(b - a, c - a) / (b - a).norm());
}
// returns true if point p is on line segment ab
bool is_point_on_seg(PT a, PT b, PT p) {
 if (fabs(cross(p - b, a - b)) < eps) {
  if (p.x < min(a.x, b.x) - eps || p.x > max(a.x, b.x) +
      eps) return false;
  if (p.y < min(a.y, b.y) - eps || p.y > max(a.y, b.y) +
      eps) return false;
  return true;
 }
 return false;
}
// minimum distance point from point c to segment ab that
     lies on segment ab
PT project_from_point_to_seg(PT a, PT b, PT c) {
 double r = dist2(a, b);
 if (sign(r) == 0) return a;
 r = dot(c - a, b - a) / r;
 if (r < 0) return a;
 if (r > 1) return b;
 return a + (b - a) * r;
}
// minimum distance from point c to segment ab
double dist_from_point_to_seg(PT a, PT b, PT c) {
 return dist(c, project_from_point_to_seg(a, b, c));
}
// 0 if not parallel, 1 if parallel, 2 if collinear
int is_parallel(PT a, PT b, PT c, PT d) {
 double k = fabs(cross(b - a, d - c));
 if (k < eps){
  if (fabs(cross(a - b, a - c)) < eps && fabs(cross(c - d
      , c - a)) < eps) return 2;
  else return 1;
 }
 else return 0;
```

```
}
// check if two lines are same
bool are_lines_same(PT a, PT b, PT c, PT d) {
 if (fabs(cross(a - c, c - d)) < eps && fabs(cross(b - c,
      c - d)) < eps) return true;
 return false;
}
// bisector vector of <abc
PT angle_bisector(PT &a, PT &b, PT &c){
 PT p = a - b, q = c - b;
 return p + q * sqrt(dot(p, p) / dot(q, q));
}
// 1 if point is ccw to the line, 2 if point is cw to the
     line, 3 if point is on the line
int point_line_relation(PT a, PT b, PT p) {
 int c = sign(cross(p - a, b - a));
 if (c < 0) return 1;
 if (c > 0) return 2;
 return 3;
}
// intersection point between ab and cd assuming unique
    intersection exists
bool line_line_intersection(PT a, PT b, PT c, PT d, PT &
    ans) {
 double a1 = a.y - b.y, b1 = b.x - a.x, c1 = cross(a, b);
 double a2 = c.y - d.y, b2 = d.x - c.x, c2 = cross(c, d);
 double det = a1 * b2 - a2 * b1;
 if (det == 0) return 0;
 ans = PT((b1 * c2 - b2 * c1) / det, (c1 * a2 - a1 * c2)
     / det);
 return 1;
}
// intersection point between segment ab and segment cd
    assuming unique intersection exists
bool seg_seg_intersection(PT a, PT b, PT c, PT d, PT &ans
    ) {
 double oa = cross2(c, d, a), ob = cross2(c, d, b);
 double oc = cross2(a, b, c), od = cross2(a, b, d);
 if (oa * ob < 0 && oc * od < 0){
  ans = (a * ob - b * oa) / (ob - oa);
  return 1;
 }
 else return 0;
}
// intersection point between segment ab and segment cd
    assuming unique intersection may not exists
// se.size()==0 means no intersection
// se.size()==1 means one intersection
// se.size()==2 means range intersection
```

```
set<PT> seg_seg_intersection_inside(PT a, PT b, PT c, PT
    d) {
 PT ans;
 if (seg_seg_intersection(a, b, c, d, ans)) return {ans};
 set<PT> se;
 if (is_point_on_seg(c, d, a)) se.insert(a);
 if (is_point_on_seg(c, d, b)) se.insert(b);
 if (is_point_on_seg(a, b, c)) se.insert(c);
 if (is_point_on_seg(a, b, d)) se.insert(d);
 return se;
}
// intersection between segment ab and line cd
// 0 if do not intersect, 1 if proper intersect, 2 if
    segment intersect
int seg_line_relation(PT a, PT b, PT c, PT d) {
 double p = cross2(c, d, a);
 double q = cross2(c, d, b);
 if (sign(p) == 0 && sign(q) == 0) return 2;
 else if (p * q < 0) return 1;
 else return 0;
}
// intersection between segament ab and line cd assuming
    unique intersection exists
bool seg_line_intersection(PT a, PT b, PT c, PT d, PT &
    ans) {
 bool k = seg_line_relation(a, b, c, d);
 assert(k != 2);
 if (k) line_line_intersection(a, b, c, d, ans);
 return k;
}
// minimum distance from segment ab to segment cd
double dist_from_seg_to_seg(PT a, PT b, PT c, PT d) {
 PT dummy;
 if (seg_seg_intersection(a, b, c, d, dummy)) return 0.0;
 else return min({dist_from_point_to_seg(a, b, c),
     dist_from_point_to_seg(a, b, d),
  dist_from_point_to_seg(c, d, a), dist_from_point_to_seg
      (c, d, b)});
}
// minimum distance from point c to ray (starting point a
     and direction vector b)
double dist_from_point_to_ray(PT a, PT b, PT c) {
 b = a + b;
 double r = dot(c - a, b - a);
 if (r < 0.0) return dist(c, a);
 return dist_from_point_to_line(a, b, c);
}
// starting point as and direction vector ad
bool ray_ray_intersection(PT as, PT ad, PT bs, PT bd) {
 double dx = bs.x - as.x, dy = bs.y - as.y;
 double det = bd.x * ad.y - bd.y * ad.x;
 if (fabs(det) < eps) return 0;
 double u = (dy * bd.x - dx * bd.y) / det;
 double v = (dy * ad.x - dx * ad.y) / det;
 if (sign(u) >= 0 && sign(v) >= 0) return 1;
 else return 0;
}
double ray_ray_distance(PT as, PT ad, PT bs, PT bd) {
 if (ray_ray_intersection(as, ad, bs, bd)) return 0.0;
 double ans = dist_from_point_to_ray(as, ad, bs);
 ans = min(ans, dist_from_point_to_ray(bs, bd, as));
 return ans;
}
```

3.2 Point

```
const int N = 3e5 + 9;

const double inf = 1e100;
const double eps = 1e-9;
const double PI = acos((double)-1.0);
int sign(double x) { return (x > eps) - (x < -eps); }
struct PT {
 double x, y;
 PT() { x = 0, y = 0; }
 PT(double x, double y) : x(x), y(y) {}
 PT(const PT &p) : x(p.x), y(p.y) {}
 PT operator + (const PT &a) const { return PT(x + a.x, y
     + a.y); }
 PT operator - (const PT &a) const { return PT(x - a.x, y
     - a.y); }
 PT operator * (const double a) const { return PT(x * a,
     y * a); }
 friend PT operator * (const double &a, const PT &b) {
     return PT(a * b.x, a * b.y); }
 PT operator / (const double a) const { return PT(x / a,
     y / a); }
 bool operator == (PT a) const { return sign(a.x - x) ==
     0 && sign(a.y - y) == 0; }
 bool operator != (PT a) const { return !(*this == a); }
 bool operator < (PT a) const { return sign(a.x - x) == 0
     ? y < a.y : x < a.x; }
 bool operator > (PT a) const { return sign(a.x - x) == 0
     ? y > a.y : x > a.x; }
 double norm() { return sqrt(x * x + y * y); }
 double norm2() { return x * x + y * y; }
 PT perp() { return PT(-y, x); }
 double arg() { return atan2(y, x); }
 PT truncate(double r) { // returns a vector with norm r
     and having same direction
  double k = norm();
  if (!sign(k)) return *this;
  r /= k;
  return PT(x * r, y * r);
 }
};
istream &operator >> (istream &in, PT &p) { return in >>
    p.x >> p.y; }
ostream &operator << (ostream &out, PT &p) { return out
    << "(" << p.x << "," << p.y << ")"; }
inline double dot(PT a, PT b) { return a.x * b.x + a.y *
    b.y; }
inline double dist2(PT a, PT b) { return dot(a - b, a - b
    ); }
inline double dist(PT a, PT b) { return sqrt(dot(a - b, a
     - b)); }
inline double cross(PT a, PT b) { return a.x * b.y - a.y
    * b.x; }
inline double cross2(PT a, PT b, PT c) { return cross(b -
     a, c - a); }
inline int orientation(PT a, PT b, PT c) { return sign(
    cross(b - a, c - a)); }
PT perp(PT a) { return PT(-a.y, a.x); }
PT rotateccw90(PT a) { return PT(-a.y, a.x); }
PT rotatecw90(PT a) { return PT(a.y, -a.x); }
PT rotateccw(PT a, double t) { return PT(a.x * cos(t) - a
    .y * sin(t), a.x * sin(t) + a.y * cos(t)); }
PT rotatecw(PT a, double t) { return PT(a.x * cos(t) + a.
    y * sin(t), -a.x * sin(t) + a.y * cos(t)); }
double SQ(double x) { return x * x; }
double rad_to_deg(double r) { return (r * 180.0 / PI); }
double deg_to_rad(double d) { return (d * PI / 180.0); }
double get_angle(PT a, PT b) {
 double costheta = dot(a, b) / a.norm() / b.norm();
 return acos(max((double)-1.0, min((double)1.0, costheta)
     ));
}
bool is_point_in_angle(PT b, PT a, PT c, PT p) { // does
    point p lie in angle <bac
 assert(orientation(a, b, c) != 0);
 if (orientation(a, c, b) < 0) swap(b, c);
 return orientation(a, c, p) >= 0 && orientation(a, b, p)
     <= 0;
}
bool half(PT p) {
 return p.y > 0.0 || (p.y == 0.0 && p.x < 0.0);
}
void polar_sort(vector<PT> &v) { // sort points in
    counterclockwise
 sort(v.begin(), v.end(), [](PT a,PT b) {
```

```
 return make_tuple(half(a), 0.0, a.norm2()) < make_tuple
     (half(b), cross(a, b), b.norm2());
 });
}
void polar_sort(vector<PT> &v, PT o) { // sort points in
    counterclockwise with respect to point o
 sort(v.begin(), v.end(), [&](PT a,PT b) {
  return make_tuple(half(a - o), 0.0, (a - o).norm2()) <
      make_tuple(half(b - o), cross(a - o, b - o), (b - o
      ).norm2());
 });
}
```

3.3 circle

```
struct circle {
 PT p; double r;
 circle() {}
 circle(PT _p, double _r): p(_p), r(_r) {};
 // center (x, y) and radius r
 circle(double x, double y, double _r): p(PT(x, y)), r(_r
     ) {};
 // circumcircle of a triangle
 // the three points must be unique
 circle(PT a, PT b, PT c) {
  b = (a + b) * 0.5;
  c = (a + c) * 0.5;
  line_line_intersection(b, b + rotatecw90(a - b), c, c +
       rotatecw90(a - c), p);
  r = dist(a, p);
 }
 // inscribed circle of a triangle
 // pass a bool just to differentiate from circumcircle
 circle(PT a, PT b, PT c, bool t) {
  line u, v;
  double m = atan2(b.y - a.y, b.x - a.x), n = atan2(c.y -
       a.y, c.x - a.x);
  u.a = a;
  u.b = u.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
  v.a = b;
  m = atan2(a.y - b.y, a.x - b.x), n = atan2(c.y - b.y, c
      .x - b.x);
  v.b = v.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
  line_line_intersection(u.a, u.b, v.a, v.b, p);
  r = dist_from_point_to_seg(a, b, p);
 }
 bool operator == (circle v) { return p == v.p && sign(r
     - v.r) == 0; }
 double area() { return PI * r * r; }
 double circumference() { return 2.0 * PI * r; }
};
```

```
//0 if outside, 1 if on circumference, 2 if inside circle
int circle_point_relation(PT p, double r, PT b) {
 double d = dist(p, b);
 if (sign(d - r) < 0) return 2;
 if (sign(d - r) == 0) return 1;
 return 0;
}
// 0 if outside, 1 if on circumference, 2 if inside
    circle
int circle_line_relation(PT p, double r, PT a, PT b) {
 double d = dist_from_point_to_line(a, b, p);
 if (sign(d - r) < 0) return 2;
 if (sign(d - r) == 0) return 1;
 return 0;
}
//compute intersection of line through points a and b
    with
//circle centered at c with radius r > 0
vector<PT> circle_line_intersection(PT c, double r, PT a,
     PT b) {
 vector<PT> ret;
 b = b - a; a = a - c;
 double A = dot(b, b), B = dot(a, b);
 double C = dot(a, a) - r * r, D = B * B - A * C;
 if (D < -eps) return ret;
 ret.push_back(c + a + b * (-B + sqrt(D + eps)) / A);
 if (D > eps) ret.push_back(c + a + b * (-B - sqrt(D)) /
     A);
 return ret;
}
//5 - outside and do not intersect
//4 - intersect outside in one point
//3 - intersect in 2 points
//2 - intersect inside in one point
//1 - inside and do not intersect
int circle_circle_relation(PT a, double r, PT b, double R
    ) {
 double d = dist(a, b);
 if (sign(d - r - R) > 0) return 5;
 if (sign(d - r - R) == 0) return 4;
 double l = fabs(r - R);
 if (sign(d - r - R) < 0 && sign(d - l) > 0) return 3;
 if (sign(d - l) == 0) return 2;
 if (sign(d - l) < 0) return 1;
 assert(0); return -1;
}
vector<PT> circle_circle_intersection(PT a, double r, PT
    b, double R) {
 if (a == b && sign(r - R) == 0) return {PT(1e18, 1e18)};
 vector<PT> ret;
 double d = sqrt(dist2(a, b));
 if (d > r + R || d + min(r, R) < max(r, R)) return ret;
 double x = (d * d - R * R + r * r) / (2 * d);
 double y = sqrt(r * r - x * x);
 PT v = (b - a) / d;
 ret.push_back(a + v * x + rotateccw90(v) * y);
 if (y > 0) ret.push_back(a + v * x - rotateccw90(v) * y)
     ;
 return ret;
}
// returns two circle c1, c2 through points a, b and of
    radius r
// 0 if there is no such circle, 1 if one circle, 2 if
    two circle
int get_circle(PT a, PT b, double r, circle &c1, circle &
    c2) {
 vector<PT> v = circle_circle_intersection(a, r, b, r);
 int t = v.size();
 if (!t) return 0;
 c1.p = v[0], c1.r = r;
 if (t == 2) c2.p = v[1], c2.r = r;
 return t;
}
// returns two circle c1, c2 which is tangent to line u,
    goes through
// point q and has radius r1; 0 for no circle, 1 if c1 =
    c2 , 2 if c1 != c2
int get_circle(line u, PT q, double r1, circle &c1,
    circle &c2) {
 double d = dist_from_point_to_line(u.a, u.b, q);
 if (sign(d - r1 * 2.0) > 0) return 0;
 if (sign(d) == 0) {
  cout << u.v.x << ' ' << u.v.y << '\n';
  c1.p = q + rotateccw90(u.v).truncate(r1);
  c2.p = q + rotatecw90(u.v).truncate(r1);
  c1.r = c2.r = r1;
  return 2;
 }
 line u1 = line(u.a + rotateccw90(u.v).truncate(r1), u.b
     + rotateccw90(u.v).truncate(r1));
 line u2 = line(u.a + rotatecw90(u.v).truncate(r1), u.b +
      rotatecw90(u.v).truncate(r1));
 circle cc = circle(q, r1);
 PT p1, p2; vector<PT> v;
 v = circle_line_intersection(q, r1, u1.a, u1.b);
 if (!v.size()) v = circle_line_intersection(q, r1, u2.a,
      u2.b);
 v.push_back(v[0]);
 p1 = v[0], p2 = v[1];
 c1 = circle(p1, r1);
```

```
 if (p1 == p2) {
  c2 = c1;
  return 1;
 }
 c2 = circle(p2, r1);
 return 2;
}
// returns the circle such that for all points w on the
    circumference of the circle
// dist(w, a) : dist(w, b) = rp : rq
// rp != rq
// https://en.wikipedia.org/wiki/Circles_of_Apollonius
circle get_apollonius_circle(PT p, PT q, double rp,
    double rq ){
 rq *= rq ;
 rp *= rp ;
 double a = rq - rp ;
 assert(sign(a));
 double g = rq * p.x - rp * q.x ; g /= a ;
 double h = rq * p.y - rp * q.y ; h /= a ;
 double c = rq * p.x * p.x - rp * q.x * q.x + rq * p.y *
     p.y - rp * q.y * q.y ;
 c /= a ;
 PT o(g, h);
 double r = g * g + h * h - c ;
 r = sqrt(r);
 return circle(o,r);
}
// returns area of intersection between two circles
double circle_circle_area(PT a, double r1, PT b, double
    r2) {
 double d = (a - b).norm();
 if(r1 + r2 < d + eps) return 0;
 if(r1 + d < r2 + eps) return PI * r1 * r1;
 if(r2 + d < r1 + eps) return PI * r2 * r2;
 double theta_1 = acos((r1 * r1 + d * d - r2 * r2) / (2 *
      r1 * d)),
  theta_2 = acos((r2 * r2 + d * d - r1 * r1)/(2 * r2 * d)
      );
 return r1 * r1 * (theta_1 - sin(2 * theta_1)/2.) + r2 *
     r2 * (theta_2 - sin(2 * theta_2)/2.);
}
// tangent lines from point q to the circle
int tangent_lines_from_point(PT p, double r, PT q, line &
    u, line &v) {
 int x = sign(dist2(p, q) - r * r);
 if (x < 0) return 0; // point in cricle
 if (x == 0) { // point on circle
  u = line(q, q + rotateccw90(q - p));
  v = u;
  return 1;
 }
 double d = dist(p, q);
 double l = r * r / d;
 double h = sqrt(r * r - l * l);
 u = line(q, p + ((q - p).truncate(l) + (rotateccw90(q -
     p).truncate(h))));
 v = line(q, p + ((q - p).truncate(l) + (rotatecw90(q - p
     ).truncate(h))));
 return 2;
}
// returns outer tangents line of two circles
// if inner == 1 it returns inner tangent lines
int tangents_lines_from_circle(PT c1, double r1, PT c2,
    double r2, bool inner, line &u, line &v) {
 if (inner) r2 = -r2;
 PT d = c2 - c1;
 double dr = r1 - r2, d2 = d.norm2(), h2 = d2 - dr * dr;
 if (d2 == 0 || h2 < 0) {
  assert(h2 != 0);
  return 0;
 }
 vector<pair<PT, PT>>out;
 for (int tmp: {- 1, 1}) {
  PT v = (d * dr + rotateccw90(d) * sqrt(h2) * tmp) / d2;
  out.push_back({c1 + v * r1, c2 + v * r2});
 }
 u = line(out[0].first, out[0].second);
 if (out.size() == 2) v = line(out[1].first, out[1].
     second);
 return 1 + (h2 > 0);
}
// O(n^2 log n)
// https://vjudge.net/problem/UVA-12056
struct CircleUnion {
 int n;
 double x[2020], y[2020], r[2020];
 int covered[2020];
 vector<pair<double, double> > seg, cover;
 double arc, pol;
 inline int sign(double x) {return x < -eps ? -1 : x >
     eps;}
 inline int sign(double x, double y) {return sign(x - y)
     ;}
 inline double SQ(const double x) {return x * x;}
 inline double dist(double x1, double y1, double x2,
     double y2) {return sqrt(SQ(x1 - x2) + SQ(y1 - y2));}
 inline double angle(double A, double B, double C) {
  double val = (SQ(A) + SQ(B) - SQ(C)) / (2 * A * B);
  if (val < -1) val = -1;
  if (val > +1) val = +1;
  return acos(val);
 }
 CircleUnion() {
  n = 0;
  seg.clear(), cover.clear();
  arc = pol = 0;
 }
 void init() {
  n = 0;
  seg.clear(), cover.clear();
  arc = pol = 0;
 }
 void add(double xx, double yy, double rr) {
  x[n] = xx, y[n] = yy, r[n] = rr, covered[n] = 0, n++;
 }
 void getarea(int i, double lef, double rig) {
  arc += 0.5 * r[i] * r[i] * (rig - lef - sin(rig - lef))
      ;
  double x1 = x[i] + r[i] * cos(lef), y1 = y[i] + r[i] *
      sin(lef);
  double x2 = x[i] + r[i] * cos(rig), y2 = y[i] + r[i] *
      sin(rig);
  pol += x1 * y2 - x2 * y1;
 }
 double solve() {
  for (int i = 0; i < n; i++) {
   for (int j = 0; j < i; j++) {
    if (!sign(x[i] - x[j]) && !sign(y[i] - y[j]) && !sign
        (r[i] - r[j])) {
     r[i] = 0.0;
     break;
    }
   }
  }
  for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) {
    if (i != j && sign(r[j] - r[i]) >= 0 && sign(dist(x[i
        ], y[i], x[j], y[j]) - (r[j] - r[i])) <= 0) {
     covered[i] = 1;
     break;
    }
   }
  }
  for (int i = 0; i < n; i++) {
   if (sign(r[i]) && !covered[i]) {
    seg.clear();
    for (int j = 0; j < n; j++) {
     if (i != j) {
      double d = dist(x[i], y[i], x[j], y[j]);
```

```
    if (sign(d - (r[j] + r[i])) >= 0 || sign(d - abs(r[
        j] - r[i])) <= 0) {
     continue;
    }
    double alpha = atan2(y[j] - y[i], x[j] - x[i]);
    double beta = angle(r[i], d, r[j]);
    pair<double, double> tmp(alpha - beta, alpha + beta
        );
    if (sign(tmp.first) <= 0 && sign(tmp.second) <= 0)
        {
     seg.push_back(pair<double, double>(2 * PI + tmp.
         first, 2 * PI + tmp.second));
    }
    else if (sign(tmp.first) < 0) {
     seg.push_back(pair<double, double>(2 * PI + tmp.
         first, 2 * PI));
     seg.push_back(pair<double, double>(0, tmp.second));
    }
    else {
     seg.push_back(tmp);
    }
   }
  }
  sort(seg.begin(), seg.end());
  double rig = 0;
  for (vector<pair<double, double> >::iterator iter =
      seg.begin(); iter != seg.end(); iter++) {
   if (sign(rig - iter->first) >= 0) {
    rig = max(rig, iter->second);
   }
   else {
    getarea(i, rig, iter->first);
    rig = iter->second;
   }
  }
  if (!sign(rig)) {
   arc += r[i] * r[i] * PI;
  }
  else {
   getarea(i, rig, 2 * PI);
  }
 }
 return pol / 2.0 + arc;
}
} CU;
```

3.4 polygon

```
double area_of_triangle(PT a, PT b, PT c) {
   return fabs(cross(b - a, c - a) * 0.5);
}
// -1 if strictly inside, 0 if on the polygon, 1 if
    strictly outside
int is_point_in_triangle(PT a, PT b, PT c, PT p) {
   if (sign(cross(b - a,c - a)) < 0) swap(b, c);
   int c1 = sign(cross(b - a,p - a));
   int c2 = sign(cross(c - b,p - b));
   int c3 = sign(cross(a - c,p - c));
   if (c1<0 || c2<0 || c3 < 0) return 1;
   if (c1 + c2 + c3 != 3) return 0;
   return -1;
}
double perimeter(vector<PT> &p) {
   double ans=0; int n = p.size();
   for (int i = 0; i < n; i++) ans += dist(p[i], p[(i +
       1) % n]);
   return ans;
}
double area(vector<PT> &p) {
   double ans = 0; int n = p.size();
   for (int i = 0; i < n; i++) ans += cross(p[i], p[(i +
       1) % n]);
   return fabs(ans) * 0.5;
}
PT centroid(vector<PT> &p) {
   int n = p.size(); PT c(0, 0);
   double sum = 0;
   for (int i = 0; i < n; i++) sum += cross(p[i], p[(i +
       1) % n]);
   double scale = 3.0 * sum;
   for (int i = 0; i < n; i++) {
      int j = (i + 1) % n;
      c = c + (p[i] + p[j]) * cross(p[i], p[j]);
   }
   return c / scale;
}
bool get_direction(vector<PT> &p) {
   double ans = 0; int n = p.size();
   for (int i = 0; i < n; i++) ans += cross(p[i], p[(i +
       1) % n]);
   if (sign(ans) > 0) return 1;
   return 0;
}
PT geometric_median(vector<PT> p) {
   auto tot_dist = [&](PT z) {
      double res = 0;
      for (int i = 0; i < p.size(); i++) res += dist(p[i
          ], z);
      return res;
   };
   auto findY = [&](double x) {
      double yl = -1e5, yr = 1e5;
      for (int i = 0; i < 60; i++) {
         double ym1 = yl + (yr - yl) / 3;
         double ym2 = yr - (yr - yl) / 3;
         double d1 = tot_dist(PT(x, ym1));
         double d2 = tot_dist(PT(x, ym2));
         if (d1 < d2) yr = ym2;
         else yl = ym1;
      }
      return pair<double, double> (yl, tot_dist(PT(x, yl
          )));
   };
   double xl = -1e5, xr = 1e5;
   for (int i = 0; i < 60; i++) {
      double xm1 = xl + (xr - xl) / 3;
      double xm2 = xr - (xr - xl) / 3;
      double y1, d1, y2, d2;
      auto z = findY(xm1); y1 = z.first; d1 = z.second;
      z = findY(xm2); y2 = z.first; d2 = z.second;
      if (d1 < d2) xr = xm2;
      else xl = xm1;
   }
   return {xl, findY(xl).first };
}
vector<PT> convex_hull(vector<PT> &p) {
   if (p.size() <= 1) return p;
   vector<PT> v = p;
   sort(v.begin(), v.end());
   vector<PT> up, dn;
   for (auto& p : v) {
      while (up.size() > 1 && orientation(up[up.size() -
          2], up.back(), p) >= 0) {
         up.pop_back();
      }
      while (dn.size() > 1 && orientation(dn[dn.size() -
          2], dn.back(), p) <= 0) {
         dn.pop_back();
      }
      up.push_back(p);
      dn.push_back(p);
   }
   v = dn;
   if (v.size() > 1) v.pop_back();
   reverse(up.begin(), up.end());
   up.pop_back();
   for (auto& p : up) {
      v.push_back(p);
   }
   if (v.size() == 2 && v[0] == v[1]) v.pop_back();
```

```
    return v;
}
 //checks if convex or not
bool is_convex(vector<PT> &p) {
    bool s[3]; s[0] = s[1] = s[2] = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        int k = (j + 1) % n;
        s[sign(cross(p[j] - p[i], p[k] - p[i])) + 1] = 1;
        if (s[0] && s[2]) return 0;
    }
    return 1;
}
// -1 if strictly inside, 0 if on the polygon, 1 if
    strictly outside
// it must be strictly convex, otherwise make it strictly
     convex first
int is_point_in_convex(vector<PT> &p, const PT& x) { // O
    (log n)
    int n = p.size(); assert(n >= 3);
    int a = orientation(p[0], p[1], x), b = orientation(p
        [0], p[n - 1], x);
    if (a < 0 || b > 0) return 1;
    int l = 1, r = n - 1;
    while (l + 1 < r) {
        int mid = l + r >> 1;
        if (orientation(p[0], p[mid], x) >= 0) l = mid;
        else r = mid;
    }
    int k = orientation(p[l], p[r], x);
    if (k <= 0) return -k;
    if (l == 1 && a == 0) return 0;
    if (r == n - 1 && b == 0) return 0;
    return -1;
}
bool is_point_on_polygon(vector<PT> &p, const PT& z) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        if (is_point_on_seg(p[i], p[(i + 1) % n], z))
            return 1;
    }
    return 0;
}
int is_point_in_polygon(vector<PT> &p, const PT& z) { //
    O(n)
    int k = winding_number(p, z);
    return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}
```

```
// maximum distance from any point on the perimeter to
    another point on the perimeter
double diameter(vector<PT> &p) {
    int n = (int)p.size();
    if (n == 1) return 0;
    if (n == 2) return dist(p[0], p[1]);
    double ans = 0;
    int i = 0, j = 1;
    while (i < n) {
        while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n]
            - p[j]) >= 0) {
            ans = max(ans, dist2(p[i], p[j]));
            j = (j + 1) % n;
        }
        ans = max(ans, dist2(p[i], p[j]));
        i++;
    }
    return sqrt(ans);
}
// given n points, find the minimum enclosing circle of
    the points
// call convex_hull() before this for faster solution
// expected O(n)
circle minimum_enclosing_circle(vector<PT> &p) {
    random_shuffle(p.begin(), p.end());
    int n = p.size();
    circle c(p[0], 0);
    for (int i = 1; i < n; i++) {
        if (sign(dist(c.p, p[i]) - c.r) > 0) {
            c = circle(p[i], 0);
            for (int j = 0; j < i; j++) {
                if (sign(dist(c.p, p[j]) - c.r) > 0) {
                    c = circle((p[i] + p[j]) / 2, dist(p[i
                        ], p[j]) / 2);
                    for (int k = 0; k < j; k++) {
                        if (sign(dist(c.p, p[k]) - c.r) > 0)
                            {
                            c = circle(p[i], p[j], p[k]);
                        }
                    }
                }
            }
        }
    }
    return c;
}
// returns a vector with the vertices of a polygon with
    everything
// to the left of the line going from a to b cut away.
vector<PT> cut(vector<PT> &p, PT a, PT b) {
    vector<PT> ans;
    int n = (int)p.size();
    for (int i = 0; i < n; i++) {
        double c1 = cross(b - a, p[i] - a);
        double c2 = cross(b - a, p[(i + 1) % n] - a);
        if (sign(c1) >= 0) ans.push_back(p[i]);
        if (sign(c1 * c2) < 0) {
            if (!is_parallel(p[i], p[(i + 1) % n], a, b))
                {
                PT tmp; line_line_intersection(p[i], p[(i +
                    1) % n], a, b, tmp);
                ans.push_back(tmp);
            }
        }
    }
    return ans;
}
// not necessarily convex, boundary is included in the
    intersection
// returns total intersected length
// it returns the sum of the lengths of the portions of
    the line that are inside the polygon
double polygon_line_intersection(vector<PT> p, PT a, PT b
    ) {
    int n = p.size();
    p.push_back(p[0]);
    line l = line(a, b);
    double ans = 0.0;
    vector< pair<double, int> > vec;
    for (int i = 0; i < n; i++) {
        int s1 = orientation(a, b, p[i]);
        int s2 = orientation(a, b, p[i + 1]);
        if (s1 == s2) continue;
        line t = line(p[i], p[i + 1]);
        PT inter = (t.v * l.c - l.v * t.c) / cross(l.v, t.
            v);
        double tmp = dot(inter, l.v);
        int f;
        if (s1 > s2) f = s1 && s2 ? 2 : 1;
        else f = s1 && s2 ? -2 : -1;
        vec.push_back(make_pair((f > 0 ? tmp - eps : tmp +
            eps), f)); // keep eps very small like 1e-12
    }
    sort(vec.begin(), vec.end());
    for (int i = 0, j = 0; i + 1 < (int)vec.size(); i++){
        j += vec[i].second;
        if (j) ans += vec[i + 1].first - vec[i].first; //
            if this portion is inside the polygon
        // else ans = 0; // if we want the maximum
            intersected length which is totally inside the
```

```
        polygon, uncomment this and take the maximum
        of ans
    }
    ans = ans / sqrt(dot(l.v, l.v));
    p.pop_back();
    return ans;
}
// given a convex polygon p, and a line ab and the top
    vertex of the polygon
// returns the intersection of the line with the polygon
// it returns the indices of the edges of the polygon
    that are intersected by the line
// so if it returns i, then the line intersects the edge
    (p[i], p[(i + 1) % n])
array<int, 2> convex_line_intersection(vector<PT> &p, PT
    a, PT b, int top) {
    int end_a = extreme_vertex(p, (a - b).perp(), top);
    int end_b = extreme_vertex(p, (b - a).perp(), top);
    auto cmp_l = [&](int i) { return orientation(a, p[i],
        b); };
    if (cmp_l(end_a) < 0 || cmp_l(end_b) > 0)
        return {-1, -1}; // no intersection
    array<int, 2> res;
    for (int i = 0; i < 2; i++) {
        int lo = end_b, hi = end_a, n = p.size();
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) %
                n;
            (cmp_l(m) == cmp_l(end_b) ? lo : hi) = m;
        }
        res[i] = (lo + !cmp_l(hi)) % n;
        swap(end_a, end_b);
    }
    if (res[0] == res[1]) return {res[0], -1}; // touches
        the vertex res[0]
    if (!cmp_l(res[0]) && !cmp_l(res[1]))
        switch ((res[0] - res[1] + (int)p.size() + 1) % p.
            size()) {
            case 0: return {res[0], res[0]}; // touches
                the edge (res[0], res[0] + 1)
            case 2: return {res[1], res[1]}; // touches
                the edge (res[1], res[1] + 1)
        }
    return res; // intersects the edges (res[0], res[0] +
        1) and (res[1], res[1] + 1)
}

pair<PT, int> point_poly_tangent(vector<PT> &p, PT Q, int
    dir, int l, int r) {
    while (r - l > 1) {
        int mid = (l + r) >> 1;
        bool pvs = orientation(Q, p[mid], p[mid - 1]) !=
            dir;
        bool nxt = orientation(Q, p[mid], p[mid + 1]) !=
            dir;
        if (pvs && nxt) return {p[mid], mid};
        if (!(pvs || nxt)) {
            auto p1 = point_poly_tangent(p, Q, dir, mid +
                1, r);
            auto p2 = point_poly_tangent(p, Q, dir, l, mid
                - 1);
            return orientation(Q, p1.first, p2.first) ==
                dir ? p1 : p2;
        }
        if (!pvs) {
            if (orientation(Q, p[mid], p[l]) == dir) r =
                mid - 1;
            else if (orientation(Q, p[l], p[r]) == dir) r
                = mid - 1;
            else l = mid + 1;
        }
        if (!nxt) {
            if (orientation(Q, p[mid], p[l]) == dir) l =
                mid + 1;
            else if (orientation(Q, p[l], p[r]) == dir) r
                = mid - 1;
            else l = mid + 1;
        }
    }
    pair<PT, int> ret = {p[l], l};
    for (int i = l + 1; i <= r; i++) ret = orientation(Q,
        ret.first, p[i]) != dir ? make_pair(p[i], i) :
        ret;
    return ret;
}
// (ccw, cw) tangents from a point that is outside this
    convex polygon
// returns indexes of the points
// ccw means the tangent from Q to that point is in the
    same direction as the polygon ccw direction
pair<int, int> tangents_from_point_to_polygon(vector<PT>
    &p, PT Q){
    int ccw = point_poly_tangent(p, Q, 1, 0, (int)p.size
        () - 1).second;
    int cw = point_poly_tangent(p, Q, -1, 0, (int)p.size
        () - 1).second;
    return make_pair(ccw, cw);
}

// minimum distance from a point to a convex polygon
// it assumes point lie strictly outside the polygon
double dist_from_point_to_polygon(vector<PT> &p, PT z) {
    double ans = inf;
    int n = p.size();
    if (n <= 3) {
        for(int i = 0; i < n; i++) ans = min(ans,
            dist_from_point_to_seg(p[i], p[(i + 1) % n], z
            ));
        return ans;
    }
    auto [r, l] = tangents_from_point_to_polygon(p, z);
    if(l > r) r += n;
    while (l < r) {
        int mid = (l + r) >> 1;
        double left = dist2(p[mid % n], z), right= dist2(p
            [(mid + 1) % n], z);
        ans = min({ans, left, right});
        if(left < right) r = mid;
        else l = mid + 1;
    }
    ans = sqrt(ans);
    ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l
        + 1) % n], z));
    ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l
        - 1 + n) % n], z));
    return ans;
}
// minimum distance from convex polygon p to line ab
// returns 0 is it intersects with the polygon
// top - upper right vertex
double dist_from_polygon_to_line(vector<PT> &p, PT a, PT
    b, int top) { //O(log n)
    PT orth = (b - a).perp();
    if (orientation(a, b, p[0]) > 0) orth = (a - b).perp
        ();
    int id = extreme_vertex(p, orth, top);
    if (dot(p[id] - a, orth) > 0) return 0.0; //if orth
        and a are in the same half of the line, then poly
        and line intersects
    return dist_from_point_to_line(a, b, p[id]); //does
        not intersect
}
// minimum distance from a convex polygon to another
    convex polygon
// the polygon doesnot overlap or touch
// tested in https://toph.co/p/the-wall
double dist_from_polygon_to_polygon(vector<PT> &p1,
    vector<PT> &p2) { // O(n log n)
    double ans = inf;
    for (int i = 0; i < p1.size(); i++) {
```

```
        ans = min(ans, dist_from_point_to_polygon(p2, p1[i
            ]));
    }
    for (int i = 0; i < p2.size(); i++) {
        ans = min(ans, dist_from_point_to_polygon(p1, p2[i
            ]));
    }
    return ans;
}
// maximum distance from a convex polygon to another
    convex polygon
double maximum_dist_from_polygon_to_polygon(vector<PT> &u
    , vector<PT> &v){ //O(n)
    int n = (int)u.size(), m = (int)v.size();
    double ans = 0;
    if (n < 3 || m < 3) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) ans = max(ans,
                dist2(u[i], v[j]));
        }
        return sqrt(ans);
    }
    if (u[0].x > v[0].x) swap(n, m), swap(u, v);
    int i = 0, j = 0, step = n + m + 10;
    while (j + 1 < m && v[j].x < v[j + 1].x) j++ ;
    while (step--) {
        if (cross(u[(i + 1)%n] - u[i], v[(j + 1)%m] - v[j
            ]) >= 0) j = (j + 1) % m;
        else i = (i + 1) % n;
        ans = max(ans, dist2(u[i], v[j]));
    }
    return sqrt(ans);
}

// calculates the area of the union of n polygons (not
    necessarily convex).
// the points within each polygon must be given in CCW
    order.
// complexity: O(N^2), where N is the total number of
    points
double rat(PT a, PT b, PT p) {
        return !sign(a.x - b.x) ? (p.y - a.y) / (b.y - a.y
            ) : (p.x - a.x) / (b.x - a.x);
 };
double polygon_union(vector<vector<PT>> &p) {
    int n = p.size();
    double ans=0;
    for(int i = 0; i < n; ++i) {
        for (int v = 0; v < (int)p[i].size(); ++v) {
            PT a = p[i][v], b = p[i][(v + 1) % p[i].size()
                ];
            vector<pair<double, int>> segs;
            segs.emplace_back(0, 0), segs.emplace_back(1,
                0);
            for(int j = 0; j < n; ++j) {
                if(i != j) {
                    for(size_t u = 0; u < p[j].size(); ++u)
                        {
                        PT c = p[j][u], d = p[j][(u + 1) % p
                            [j].size()];
                        int sc = sign(cross(b - a, c - a)),
                            sd = sign(cross(b - a, d - a));
                        if(!sc && !sd) {
                            if(sign(dot(b - a, d - c)) > 0 &&
                                i > j) {
                                segs.emplace_back(rat(a, b, c
                                    ), 1), segs.emplace_back(
                                    rat(a, b, d), -1);
                            }
                        }
                        else {
                            double sa = cross(d - c, a - c),
                                sb = cross(d - c, b - c);
                            if(sc >= 0 && sd < 0) segs.
                                emplace_back(sa / (sa - sb),
                                1);
                            else if(sc < 0 && sd >= 0) segs.
                                emplace_back(sa / (sa - sb),
                                -1);
                        }
                    }
                }
            }
            sort(segs.begin(), segs.end());
            double pre = min(max(segs[0].first, 0.0), 1.0)
                , now, sum = 0;
            int cnt = segs[0].second;
            for(int j = 1; j < segs.size(); ++j) {
                now = min(max(segs[j].first, 0.0), 1.0);
                if (!cnt) sum += now - pre;
                cnt += segs[j].second;
                pre = now;
            }
            ans += cross(a, b) * sum;
        }
    }
    return ans * 0.5;
}

// returns the area of the intersection of the circle
    with center c and radius r
// and the triangle formed by the points c, a, b
double _triangle_circle_intersection(PT c, double r, PT a
    , PT b) {
    double sd1 = dist2(c, a), sd2 = dist2(c, b);
    if(sd1 > sd2) swap(a, b), swap(sd1, sd2);
    double sd = dist2(a, b);
    double d1 = sqrtl(sd1), d2 = sqrtl(sd2), d = sqrt(sd)
        ;
    double x = abs(sd2 - sd - sd1) / (2 * d);
    double h = sqrtl(sd1 - x * x);
    if(r >= d2) return h * d / 2;
    double area = 0;
    if(sd + sd1 < sd2) {
        if(r < d1) area = r * r * (acos(h / d2) - acos(h /
            d1)) / 2;
        else {
            area = r * r * ( acos(h / d2) - acos(h / r)) /
                2;
            double y = sqrtl(r * r - h * h);
            area += h * (y - x) / 2;
        }
    }
    else {
        if(r < h) area = r * r * (acos(h / d2) + acos(h /
            d1)) / 2;
        else {
            area += r * r * (acos(h / d2) - acos(h / r)) /
                2;
            double y = sqrtl(r * r - h * h);
            area += h * y / 2;
            if(r < d1) {
                area += r * r * (acos(h / d1) - acos(h / r)
                    ) / 2;
                area += h * y / 2;
            }
            else area += h * x / 2;
        }
    }
    return area;
}
// intersection between a simple polygon and a circle
double polygon_circle_intersection(vector<PT> &v, PT p,
    double r) {
    int n = v.size();
    double ans = 0.00;
    PT org = {0, 0};
    for(int i = 0; i < n; i++) {
        int x = orientation(p, v[i], v[(i + 1) % n]);
```

```
        if(x == 0) continue;
        double area = _triangle_circle_intersection(org, r
            , v[i] - p, v[(i + 1) % n] - p);
        if (x < 0) ans -= area;
        else ans += area;
    }
    return abs(ans);
}
// find a circle of radius r that contains as many points
     as possible
// O(n^2 log n);
double maximum_circle_cover(vector<PT> p, double r,
    circle &c) {
    int n = p.size();
    int ans = 0;
    int id = 0; double th = 0;
    for (int i = 0; i < n; ++i) {
        // maximum circle cover when the circle goes
            through this point
        vector<pair<double, int>> events = {{-PI, +1}, {PI
            , -1}};
        for (int j = 0; j < n; ++j) {
            if (j == i) continue;
            double d = dist(p[i], p[j]);
            if (d > r * 2) continue;
            double dir = (p[j] - p[i]).arg();
            double ang = acos(d / 2 / r);
            double st = dir - ang, ed = dir + ang;
            if (st > PI) st -= PI * 2;
            if (st <= -PI) st += PI * 2;
            if (ed > PI) ed -= PI * 2;
            if (ed <= -PI) ed += PI * 2;
            events.push_back({st - eps, +1}); // take care
                 of precisions!
            events.push_back({ed, -1});
            if (st > ed) {
                events.push_back({-PI, +1});
                events.push_back({+PI, -1});
            }
        }
        sort(events.begin(), events.end());
        int cnt = 0;
        for (auto &&e: events) {
            cnt += e.second;
            if (cnt > ans) {
                ans = cnt;
                id = i; th = e.first;
            }
        }
    }
    PT w = PT(p[id].x + r * cos(th), p[id].y + r * sin(th
        ));
    c = circle(w, r); //best_circle
    return ans;
}
// radius of the maximum inscribed circle in a convex
    polygon
double maximum_inscribed_circle(vector<PT> p) {
    int n = p.size();
    if (n <= 2) return 0;
    double l = 0, r = 20000;
    while (r - l > eps) {
        double mid = (l + r) * 0.5;
        vector<HP> h;
        const int L = 1e9;
        h.push_back(HP(PT(-L, -L), PT(L, -L)));
        h.push_back(HP(PT(L, -L), PT(L, L)));
        h.push_back(HP(PT(L, L), PT(-L, L)));
        h.push_back(HP(PT(-L, L), PT(-L, -L)));
        for (int i = 0; i < n; i++) {
            PT z = (p[(i + 1) % n] - p[i]).perp();
            z = z.truncate(mid);
            PT y = p[i] + z, q = p[(i + 1) % n] + z;
            h.push_back(HP(p[i] + z, p[(i + 1) % n] + z));
        }
        vector<PT> nw = half_plane_intersection(h);
        if (!nw.empty()) l = mid;
        else r = mid;
    }
    return l;
}
```

4 Graph

4.1 Lifting, LCA, HLD

```
using Tree = vector<vector<int>>;
int anc[B][N], sz[N], lvl[N], st[N], en[N], nxt[N], t =
    0;

void initLifting(int n) {
 for (int b = 1; b < B; b++) {
  for (int i = 0; i < n; i++) {
   anc[b][i] = anc[b - 1][anc[b - 1][i]];
  }
 }
}

int kthAncestor(int u, int k) {
 for (int b = 0; b < B; b++) {
  if (k >> b & 1) u = anc[b][u];
 }
 return u;
}

int lca(int u, int v) {
 if (lvl[u] > lvl[v]) swap(u, v);
 v = kthAncestor(v, lvl[v] - lvl[u]);

 if (u == v) return u;

 for (int b = B - 1; b >= 0; b--) {
  if (anc[b][u] != anc[b][v]) u = anc[b][u], v = anc[b][v
      ];
 }
 return anc[0][u];
}

int dis(int u, int v) {
 int g = lca(u, v);
 return lvl[u] + lvl[v] - 2 * lvl[g];
}
bool isAncestor(int u, int v) { return st[v] <= st[u] and
     en[u] <= en[v]; }

void tour(int u, int p, Tree &T) {
 st[u] = t++;
 int idx = 0;
 for (int v : T[u]) {
  if (v == p) continue;
  nxt[v] = (idx++ ? v : nxt[u]); // only for hld
  anc[0][v] = u, lvl[v] = lvl[u] + 1;
  tour(v, u, T);
 }
 en[u] = t; // [st, en] contains subtree range
}

void hld(int u, int p, Tree &T) {
 sz[u] = 1;

 int eld = 0, mx = 0, idx = 0;
 for (int i = 0; i < T[u].size(); i++) {
  int v = T[u][i];
  if (v == p) continue;
  hld(v, u, T);

  if (sz[v] > mx) mx = sz[v], eld = i;
  sz[u] += sz[v];
 }
 swap(T[u][0], T[u][eld]);
}
```

```
LL climbQuery(int u, int g) {
 LL ans = -INF;
 while (1) {
  int _u = nxt[u];
  if (isAncestor(g, _u)) _u = g;
  ans = max(ans, rmq ::query(st[_u], st[u]));

  if (_u == g) break;
  u = anc[0][_u];
 }
 return ans;
}

LL pathQuery(int u, int v) {
 int g = lca(u, v);
 return max(climbQuery(u, g), climbQuery(v, g));
}

void init(int u, Tree &T) {
 int n = T.size();
 anc[0][u] = nxt[u] = u;
 lvl[u] = 0;
 hld(u, u, T);
 tour(u, u, T);
 initLifting(n);
}
```

4.2 Virtual Tree

```
namespace lca1 {
int st[N], lvl[N];
int tbl[B][2 * N];
int t = 0;

void dfs(int u, int p, Tree &T) {
 st[u] = t;
 tbl[0][t++] = u;
 for(int v: T[u]) {
  if(v == p) continue;
  lvl[v] = lvl[u] + 1;
  dfs(v, u, T);
  tbl[0][t++] = u;
 }
}
int low(int u, int v) {
 return make_pair(lvl[u], u) < make_pair(lvl[v], v) ? u :
      v;
}

void makeTable(int n) {
 int m = 2 * n - 1;
 for(int b = 1; b < B; b++) {
  for(int i = 0; i < m; i++) {
   tbl[b][i] = low(tbl[b - 1][i], tbl[b - 1][i + (1 << b
       - 1)]);
  }
 }
}

int lca(int u, int v) {
 int l = st[u], r = st[v];
 if(l > r) swap(l, r);
 int k = __lg(r - l + 1);
 return low(tbl[k][l], tbl[k][r - (1 << k) + 1]);
}
void init(int root, Tree &T) {
 lvl[root] = 0;
 t = 0;
 dfs(root, root, T);
 makeTable(T.size());
}
}
namespace vt {
int st[N], en[N], t;
vector <int> adj[N];

void dfs(int u, int p, Tree &T) {
 st[u] = t++;
 for(int v: T[u]) if(v != p) dfs(v, u, T);
 en[u] = t++;
}
bool comp(int u, int v) {
 return st[u] < st[v];
}
bool isAncestor(int u, int p) {
 return st[p] <= st[u] and en[u] <= en[p];
}

void construct(vector <int> &nodes) {
 sort(nodes.begin(), nodes.end(), comp);
 int n = nodes.size();
 for(int i = 0; i + 1 < n; i++) {
  nodes.push_back(lca1 :: lca(nodes[i], nodes[i + 1]));
 }
 sort(nodes.begin(), nodes.end(), comp);
 nodes.erase(unique(nodes.begin(), nodes.end()), nodes.
     end());
 n = nodes.size();
 stack <int> s;
 s.push(nodes[0]);
 for(int i = 1; i < n; i++) {
  int u = nodes[i];
  while(not isAncestor(u, s.top())) s.pop();
  adj[s.top()].push_back(u);
  s.push(u);
 }
}
void clear(vector <int> &nodes) {
 for(int u: nodes) {
   adj[u].clear();
 }
}

void init(int root, Tree &T) {
 lca1 :: init(root, T);
 t = 0;
 dfs(root, root, T);
}
}
```

4.3 Bridge Tree

```
vector<vector<int>> comps;
vector<int> depth, low, id;
stack<int> st;
vector<Edge> bridges;
Graph tree;

void dfs(int u, Graph &G, int ed = -1, int d = 0) {
 low[u] = depth[u] = d;
 st.push(u);
 for (int e : G[u]) {
  if (e == ed) continue;
  int v = G(e).to(u);
  if (depth[v] == -1) dfs(v, G, e, d + 1);
  low[u] = min(low[u], low[v]);

  if (low[v] <= depth[u]) continue;
  bridges.emplace_back(u, v);
  comps.emplace_back();
  do {
   comps.back().push_back(st.top()), st.pop();
  } while (comps.back().back() != v);
 }
 if (ed == -1) {
  comps.emplace_back();
  while (!st.empty()) comps.back().push_back(st.top()),
      st.pop();
 }
}
Graph &createTree() {
```

```
 for (auto &comp : comps) {
  int idx = tree.addNode();
  for (auto &e : comp) id[e] = idx;
 }
 for (auto &[l, r] : bridges) tree.addEdge(id[l], id[r]);
 return tree;
}

void init(Graph &G) {
 int n = G.n;
 depth.assign(n, -1), id.assign(n, -1), low.resize(n);
 for (int i = 0; i < n; i++) {
  if (depth[i] == -1) dfs(i, G);
 }
}
```

5 Mahir

5.1 temp

```
vector<int> factor(ll n) {
 vector<int> ret;
 for (int i = 2; i * i <= n; i++) {
  while (n % i == 0) {
   ret.push_back(i);
   n /= i;
  }
 }
 if (n > 1) { ret.push_back(n); }
 return ret;
}
void sieve_of_eratosthenes(int n)
{
 bool is_prime[n + 1];
 memset(is_prime, true, sizeof(is_prime));
 is_prime[0] = is_prime[1] = false;
 for (int p = 2; p * p <= n; p++) {
  if (is_prime[p]) {
   for (ll i = p * p; i <= n; i += p) {
    is_prime[i] = false;
   }
  }
 }
 for (int i = 2; i <= n; i++) {
  if (is_prime[i]) {
   primes.push_back(i);
  }
 }
}
// spf start
const ll N = 2000001;
ll lp[N+1];
vector<ll> pr;
void sieve(){
 ll i,j;
 for (i=2; i<=N; ++i) {
  if (lp[i] == 0) {
   lp[i] = i;
   pr.push_back (i);
  }
  for (j=0; j<(ll)pr.size() && pr[j]<=lp[i] && i*pr[j]<=N
      ; ++j)
   lp[i*pr[j]] = pr[j];
 }
}
vector<ll> spf(ll n){
 vector<ll> pf;
 while (n != 1){
  pf.push_back(lp[n]);
  n = n / lp[n];
 }
 return pf;
}
void pre()
{
 sieve();
}
// spf end
// nCr
ll fact[N+1];
ll mod_exp(ll base, ll exp, ll mod) {
 base %= mod;
 ll res = 1;
 while (exp> 0) {
  if (exp%2==1)
   res = res*base%mod;
  base = base*base%mod;
  exp/=2;
 }
 return res;
}
ll nCr (ll n, ll r) {
 if (n<r)
  return 0ll;
 return (fact[n]*mod_exp(fact[r]*fact[n-r]%MOD,MOD-2,MOD)
     )%MOD;
}
void pre()
{
 fact[0] = 1;
 for (int i = 1; i <= N; i++)
  fact[i] = fact[i-1]*i%MOD;
}
// nCr end
ll nCr_large(ll n, ll r) {
 if (n < r || r < 0) return 0;
 ll res = 1;
 for (ll i = 0; i < r; i++) {
  res = res * ((n - i) % MOD) % MOD;
 }
 res = res * inv_fact[r] % MOD;
 return res;
}
// Modular Inverse Start
ll mod_exp(ll base, ll exp, ll mod) {
 base %= mod;
 ll res = 1;
 while (exp> 0) {
  if (exp%2==1)
   res = res*base%mod;
  base = base*base%mod;
  exp/=2;
 }
 return res;
}
ll modInv (ll a, ll MOD) {
 return mod_exp(a, MOD - 2, MOD);
}
// Modular Inverse End
//topological sort
vector<ll> topoSort(vector<vector<ll>>& adj)
{
 ll n = adj.size()-1;
 vector<ll> indegree(n+1, 0);
 queue<ll> q;
 vector<ll> list;
 // Compute indegrees
 for (ll i = 1; i <= n; i++)
 {
  for (ll next : adj[i])
   indegree[next]++;
 }
 // Add all nodes with indegree 0
 // into the queue
 for (ll i = 1; i <= n; i++)
 {
  if (indegree[i] == 0)
   q.push(i);
 }
 // 'Kahns Algorithm (BFS)
 while (!q.empty())
```

```
 {
  ll top = q.front();
  q.pop();
  list.push_back(top);
  for (ll next : adj[top])
  {
   indegree[next]--;
   if (indegree[next] == 0)
    q.push(next);
  }
 }
 return list;
}
//topological sort end
//topological sort using dfs
int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;
void dfs(int v) {
 visited[v] = true;
 for (int u : adj[v]) {
  if (!visited[u]) {
   dfs(u);
  }
 }
 ans.push_back(v);
}
void topological_sort() {
 visited.assign(n, false);
 ans.clear();
 for (int i = 0; i < n; ++i) {
  if (!visited[i]) {
   dfs(i);
  }
 }
 reverse(ans.begin(), ans.end());
}
//bipartite check
int n;
vector<vector<int>> adj;
vector<int> side(n+1, -1);
bool is_bipartite = true;
queue<int> q;
for (int st = 1; st <= n; ++st)
{
 if (side[st] == -1)
 {
  q.push(st);
  side[st] = 0;
  while (!q.empty())
  {
   int v = q.front();
   q.pop();
   for (int u : adj[v])
   {
    if (side[u] == -1)
    {
     side[u] = side[v] ^ 1;
     q.push(u);
    }
    else
    {
     is_bipartite &= (side[u] != side[v]);
    }
   }
  }
 }
}
cout << (is_bipartite ? "YES" : "NO") << endl;
// bipartite end
// Simple segment tree. Supports range query and point
    update.
struct segtree {
 ll NEUTRAL_ELEMENT = 0;
 ll size;
 vector<ll>sums;
 ll single (ll val) {
  return val;
 }
 ll merge (ll a, ll b) {
  return a+b;
 }
 void init (ll n) {
  size = 1;
  while (size<n)
   size *= 2;
  sums.assign(2*size,NEUTRAL_ELEMENT);
 }
 void build (vector<ll>&v, ll x, ll lx, ll rx) {
  if (rx-lx==1) {
   if (lx<(ll)v.size())
    sums[x] = single(v[lx]);
   return;
  }
  ll mid = (lx+rx)/2;
  build(v,2*x+1,lx,mid);
  build(v,2*x+2,mid,rx);
  sums[x] = merge(sums[2*x+1],sums[2*x+2]);
 }
 void build (vector<ll>&v) {
  build(v,0,0,size);
 }
 void set (ll i, ll val, ll x, ll lx, ll rx) {
  if (rx-lx==1) {
   sums[x] = single(val);
   return;
  }
  ll mid = (lx+rx)/2;
  if (i<mid)
   set(i,val,2*x+1,lx,mid);
  else
   set(i,val,2*x+2,mid,rx);
  sums[x] = merge(sums[2*x+1],sums[2*x+2]);
 }
 void set(ll i, ll val) {
  set(i,val,0,0,size);
 }
 ll query(ll l, ll r, ll x, ll lx, ll rx) {
  if (lx>=r || rx<=l)
   return NEUTRAL_ELEMENT;
  if (lx>=l && rx<=r)
   return sums[x];
  ll mid = (lx+rx)/2;
  ll s1 = query(l,r,2*x+1,lx,mid);
  ll s2 = query(l,r,2*x+2,mid,rx);
  return merge(s1,s2);
 }
 ll query(ll l, ll r) {
  return query(l,r,0,0,size);
 }
};
//code end
// Segment Tree with Lazy Propagation
struct segtree {
 ll size;
 vector<ll>operations,sums;
 ll NO_OP = inf;
 ll op (ll a, ll b) {
  if (b==NO_OP)
   return a;
  return b;
 }
 void apply_op (ll &a, ll b) {
  a = op(a,b);
 }
 void init (ll n) {
  size = 1;
  while (size<n)
   size *= 2;
```

```
 operations.assign(2*size,NO_OP);
 sums.assign(2*size,0);
}
void build (vector<ll>&v, ll x, ll lx, ll rx) {
 if (rx-lx==1) {
  if (lx<(ll)v.size())
   sums[x] = v[lx];
  return;
 }
 ll mid = (lx+rx)/2;
 build(v,2*x+1,lx,mid);
 build(v,2*x+2,mid,rx);
 sums[x] = (sums[2*x+1]+sums[2*x+2]);
}
void build (vector<ll>&v) {
 build(v,0,0,size);
}
void propagate (ll x, ll lx, ll rx) {
 if (rx-lx==1 || operations[x]==NO_OP)
  return;
 ll mid = (lx+rx)/2ll;
 apply_op(operations[2*x+1],operations[x]);
 apply_op(operations[2*x+2],operations[x]);
 apply_op(sums[2*x+1],operations[x]*(mid-lx));
 apply_op(sums[2*x+2],operations[x]*(rx-mid));
 operations[x] = NO_OP;
}
void modify(ll l, ll r, ll val, ll x, ll lx, ll rx) {
 propagate(x,lx,rx);
 if (lx>=r || rx<=l)
  return;
 if (lx>=l && rx<=r) {
  apply_op(operations[x],val);
  apply_op(sums[x],val*(rx-lx));
  return;
 }
 ll mid = (lx+rx)/2;
 modify(l,r,val,2*x+1,lx,mid);
 modify(l,r,val,2*x+2,mid,rx);
 sums[x] = sums[2*x+1]+sums[2*x+2];
}
void modify(ll l, ll r, ll val) {
 modify(l,r,val,0,0,size);
}
ll get (ll l, ll r, ll x, ll lx, ll rx) {
 propagate(x,lx,rx);
 if (lx>=r || rx<=l)
  return 0;
 if (lx>=l && rx<=r)
  return sums[x];
 ll mid = (lx+rx)/2ll;
 return get(l,r,2*x+1,lx,mid)+get(l,r,2*x+2,mid,rx);
}
ll get(ll l, ll r) {
 return get(l,r,0,0,size);
}
};
// Code end
string convert_base(ll n, int base) {
 if (n==0)
  return "0";
 ll power = 1;
 while (power*base<=n)
  power *= base;
 string result;
 while (power>0) {
  ll k = n/power;
  result += k + '0';
  n -= power*k;
  power /= base;
 }
 return result;
}
using T = pair<ll, ll>;
void dijkstra(ll node, vector<vector<pair<ll, ll>>>& adj,
     vector<ll>& vis, vector<ll>& dis)
{
 priority_queue<T,vector<T>,greater<T>>pq;
 dis[node] = 0;
 pq.push({0,node});
 while (!pq.empty())
 {
  ll curr = pq.top().S;
  pq.pop();
  if (vis[curr])
   continue;
  vis[curr] = 1;
  for (auto [x, d]:adj[curr])
  {
   if (dis[curr]+d<dis[x])
   {
    dis[x] = dis[curr]+d;
    pq.push({dis[x],x});
   }
  }
 }
}
//Range query point update with BIT
struct BIT {
 vector<ll> sums;
 int n;
 void init(int n) {
  sums.assign(n + 1, 0);
  this->n = n;
 }
 void build(const vector<ll> &v) {
  for (int i = 1; i <= n; ++i) {
   sums[i] += v[i - 1];
   int par = i + (i & -i);
   if (par <= n)
    sums[par] += sums[i];
  }
 }
 void update(int idx, ll x) {
  for (int i = idx; i <= n; i += (i & -i))
   sums[i] += x;
 }
 ll query(int r) {
  ll res = 0;
  for (int i = r; i > 0; i -= (i & -i))
   res += sums[i];
  return res;
 }
 ll query(int l, int r) {
  return query(r) - query(l - 1);
 }
};
// Simple segment tree. Supports range query and point
    update.
// Sparse Table
struct sparseTable {
 int n, lg;
 vector<vector<int>> t;
 void init(int n) {
  this->n = n;
  this->lg = 31 - __builtin_clz(n) + 1;
  t.assign(n, vector<int>(lg, 0));
 }
 void build(const vector<int> &v) {
  for(int i = 0; i < n; ++i) {
   t[i][0] = v[i];
  }
  for(int k = 1; k < lg; ++k) {
   for(int i = 0; i + (1 << k) - 1 < n; ++i) {
    t[i][k] = min(t[i][k - 1], t[i + (1 << (k - 1))][k -
        1]);
   }
  }
 }
 int query(int l, int r) {
```

```
  int k = 31 - __builtin_clz(r - l + 1);
  return min(t[l][k], t[r - (1 << k) + 1][k]);
 }
};
// end
// Cycle in directed graph
int n;
vector<vector<int>> adj;
vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;
bool dfs(int v) {
 color[v] = 1;
 for (int u : adj[v]) {
  if (color[u] == 0) {
   parent[u] = v;
   if (dfs(u))
    return true;
  } else if (color[u] == 1) {
   cycle_end = v;
   cycle_start = u;
   return true;
  }
 }
 color[v] = 2;
 return false;
}
void find_cycle() {
 color.assign(n, 0);
 parent.assign(n, -1);
 cycle_start = -1;
 for (int v = 0; v < n; v++) {
  if (color[v] == 0 && dfs(v))
   break;
 }
 if (cycle_start == -1) {
  cout << "Acyclic" << endl;
 } else {
  vector<int> cycle;
  cycle.push_back(cycle_start);
  for (int v = cycle_end; v != cycle_start; v = parent[v
      ])
   cycle.push_back(v);
  cycle.push_back(cycle_start);
  reverse(cycle.begin(), cycle.end());
  cout << "Cycle found: ";
  for (int v : cycle)
   cout << v << " ";
  cout << endl;
 }
}
// cycle in undirected graph
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;
bool dfs(int v, int par) { // passing vertex and its
    parent vertex
 visited[v] = true;
 for (int u : adj[v]) {
  if(u == par) continue; // skipping edge to parent
      vertex
  if (visited[u]) {
   cycle_end = v;
   cycle_start = u;
   return true;
  }
  parent[u] = v;
  if (dfs(u, parent[u]))
   return true;
 }
 return false;
}
void find_cycle() {
 visited.assign(n, false);
 parent.assign(n, -1);
 cycle_start = -1;
 for (int v = 0; v < n; v++) {
  if (!visited[v] && dfs(v, parent[v]))
   break;
 }
 if (cycle_start == -1) {
  cout << "Acyclic" << endl;
 } else {
  vector<int> cycle;
  cycle.push_back(cycle_start);
  for (int v = cycle_end; v != cycle_start; v = parent[v
      ])
   cycle.push_back(v);
  cycle.push_back(cycle_start);
  cout << "Cycle found: ";
  for (int v : cycle)
   cout << v << " ";
  cout << endl;
 }
}
// MST Kruskal
vector<int> parent, rank;
void make_set(int v) {
 parent[v] = v;
 rank[v] = 0;
}
int find_set(int v) {
 if (v == parent[v])
  return v;
 return parent[v] = find_set(parent[v]);
}
void union_sets(int a, int b) {
 a = find_set(a);
 b = find_set(b);
 if (a != b) {
  if (rank[a] < rank[b])
   swap(a, b);
  parent[b] = a;
  if (rank[a] == rank[b])
   rank[a]++;
 }
}
struct Edge {
 int u, v, weight;
 bool operator<(Edge const& other) {
  return weight < other.weight;
 }
};
int n;
vector<Edge> edges;
int cost = 0;
vector<Edge> result;
parent.resize(n);
rank.resize(n);
for (int i = 0; i < n; i++)
 make_set(i);
sort(edges.begin(), edges.end());
for (Edge e : edges) {
 if (find_set(e.u) != find_set(e.v)) {
  cost += e.weight;
  result.push_back(e);
  union_sets(e.u, e.v);
 }
}
// Prim's Algorithm (MST)
using T = pair<ll, int>;
void solve() {
 int n, m;
 cin >> n >> m;
 vector<vector<T>> adj(n + 1);
 while (m--) {
  int a, b;
  ll w;
```

```
  cin >> a >> b >> w;
  adj[a].eb(w, b);
  adj[b].eb(w, a);
 }
 vector<bool> selected(n + 1, 0);
 priority_queue<T, vector<T>, greater<T>> pq;
 ll minWeight = 0;
 pq.emplace(0, 1);
 while (!pq.empty()) {
  auto [d, u] = pq.top();
  pq.pop();
  if (selected[u])
   continue;
  selected[u] = 1;
  minWeight += d;
  for (auto &[nd, v] : adj[u]) {
   if (!selected[v])
    pq.emplace(nd, v);
  }
 }
 if (count(all(selected), 0) != 1)
  cout << "IMPOSSIBLE";
 else
  cout << minWeight;
}
// LIS
int lis(vector<int> const& a) {
 int n = a.size();
 const int INF = 1e9;
 vector<int> d(n+1, INF);
 d[0] = -INF;
 for (int i = 0; i < n; i++) {
  int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin
      ();
  if (d[l-1] < a[i] && a[i] < d[l])
   d[l] = a[i];
 }
 int ans = 0;
 for (int l = 0; l <= n; l++) {
  if (d[l] < INF)
   ans = l;
 }
 return ans;
}
// Strongly Connected Components
// Kosaraju's Algorithm
// Reminder: This code is 0-indexed!!!!
constexpr ll inf = 1e18;
const ll N = 5e5 + 5;
const ll M = 505;
```

```
const ll MOD = 998244353; // 998244353 // 1000000007
vector<bool> vis;
void dfs (int node, vector<int> adj[], vector<int> &order
    ) {
 vis[node] = 1;
 for (auto &c : adj[node]) {
  if (!vis[c])
   dfs(c, adj, order);
 }
 order.pb(node);
}
void pre() {
}
void init (ll n) {
}
void solve() {
 int n, m;
 cin >> n >> m;
 vector<int> adj[n], adj_rev[n];
 for (int i = 0; i < m; ++i) {
  int u, v;
  cin >> u >> v;
  adj[u].pb(v);
  adj_rev[v].pb(u);
 }
 vis.assign(n, false);
 vector<int> order;
 for (int i = 0; i < n; ++i) {
  if (!vis[i]) {
   dfs(i, adj, order);
  }
 }
 reverse(all(order));
 vis.assign(n, false);
 vector<vector<int>> SCCs;
 for (auto &c : order) {
  if (!vis[c]) {
   vector<int> component;
   dfs(c, adj_rev, component);
   SCCs.pb(component);
  }
 }
 cout << SCCs.size() << "\n";
 for (auto &c : SCCs) {
  cout << c.size() << " ";
  for (auto &d : c)
   cout << d << " ";
  cout << "\n";
 }
}
```

```
// 2-SAT
struct twosat {
 int n; // total size combining +, -. must be even.
 vector<vector<int>> g, gt;
 vector<bool> vis, res;
 vector<int> comp;
 stack<int> ts;
 twosat(int vars = 0) {
  n = vars << 1;
  g.resize(n);
  gt.resize(n);
 }
 // zero indexed, be careful
 // if you want to force variable a to be true in OR or
     XOR combination
 // add addOR (a,1,a,1);
 // if you want to force variable a to be false in OR or
     XOR combination
 // add addOR (a,0,a,0);
 // (x_a or (not x_b))-> af=1,bf=0
 void addOR(int a, bool af, int b, bool bf) {
  a += a + (af ^ 1);
  b += b + (bf ^ 1);
  g[a ^ 1].push_back(b); // !a => b
  g[b ^ 1].push_back(a); // !b => a
  gt[b].push_back(a ^ 1);
  gt[a].push_back(b ^ 1);
 }
 // (!x_a xor !x_b)-> af=0, bf=0
 void addXOR(int a, bool af, int b, bool bf) {
  addOR(a, af, b, bf);
  addOR(a, !af, b, !bf);
 }
 void _add(int a, bool af, int b, bool bf) {
  a += a + (af ^ 1);
  b += b + (bf ^ 1);
  g[a].push_back(b);
  gt[b].push_back(a);
 }
 // add this type of condition ->
 // add(a,af,b,bf) means if a is af then b must need to
     be bf
 void add(int a, bool af, int b, bool bf) {
  _add(a, af, b, bf);
  _add(b, !bf, a, !af);
 }
 void dfs1(int u) {
  vis[u] = true;
  for (int v : g[u]) {
   if (!vis[v]) dfs1(v);
```

```
  }
  ts.push(u);
 }
 void dfs2(int u, int c) {
  comp[u] = c;
  for (int v : gt[u]) {
   if (comp[v] == -1) dfs2(v, c);
  }
 }
 bool ok() {
  vis.assign(n, false);
  for (int i = 0; i < n; ++i) {
   if (!vis[i]) dfs1(i);
  }
  int scc = 0;
  comp.assign(n, -1);
  while (!ts.empty()) {
   int u = ts.top();
   ts.pop();
   if (comp[u] == -1) dfs2(u, scc++);
  }
  res.assign(n / 2, false);
  for (int i = 0; i < n; i += 2) {
   if (comp[i] == comp[i + 1]) return false;
   res[i / 2] = (comp[i] > comp[i + 1]);
  }
  return true;
 }
};
// totient phi
void phi_1_to_n(int n) {
 vector<int> phi(n + 1);
 for (int i = 0; i <= n; i++)
  phi[i] = i;
 for (int i = 2; i <= n; i++) {
  if (phi[i] == i) {
   for (int j = i; j <= n; j += i)
    phi[j] -= phi[j] / i;
  }
 }
}
int phi(int n) {
 int result = n;
 for (int i = 2; i * i <= n; i++) {
  if (n % i == 0) {
   while (n % i == 0)
    n /= i;
   result -= result / i;
  }
 }
 if (n > 1)
  result -= result / n;
 return result;
}
// Floyd_warshall Start
ll adj[N][N], dis[N][N];
for (int i = 1; i <= n; ++i) {
 for (int j = 1; j <= n; ++j) {
  if (i==j)
   dis[i][j] = 0;
  else if (adj[i][j])
   dis[i][j] = adj[i][j];
  else
   dis[i][j] = 1e18;
 }
}
for (int k = 1; k <= n; ++k) {
 for (int i = 1; i <= n; ++i) {
  for (int j = 1; j <= n; ++j) {
   dis[i][j] = min(dis[i][j],dis[i][k]+dis[k][j]);
  }
 }
}
// Floyd_warshall End
// code to find minimum steps to visit all nodes from a
    source and go back to that source in an undirected
    tree
ll dfs(ll node, ll par) {
 ll cost = 0;
 for (auto c:adj[node]) {
  if (c==par)
   continue;
  ll temp = dfs(c,node);
  cost += 2+temp;
 }
 return cost;
}
//end
//count of inversions in an array
ll countAndMerge(vector<ll>& arr, ll l, ll m, ll r) {
 ll n1 = m - l + 1, n2 = r - m;
 vector<ll> left(n1), right(n2);
 for (ll i = 0; i < n1; i++)
  left[i] = arr[i + l];
 for (ll j = 0; j < n2; j++)
  right[j] = arr[m + 1 + j];
 ll res = 0;
 ll i = 0, j = 0, k = l;
 while (i < n1 && j < n2) {
  if (left[i] <= right[j])
   arr[k++] = left[i++];
  else {
   arr[k++] = right[j++];
   res += (n1 - i);
  }
 }
 while (i < n1)
  arr[k++] = left[i++];
 while (j < n2)
  arr[k++] = right[j++];
 return res;
}
ll countInv(vector<ll>& arr, ll l, ll r){
 ll res = 0;
 if (l < r) {
  ll m = (r + l) / 2;
  res += countInv(arr, l, m);
  res += countInv(arr, m + 1, r);
  res += countAndMerge(arr, l, m, r);
 }
 return res;
}
ll inversionCount(vector<ll>&v) {
 ll n = v.size();
 return countInv(v, 0, n - 1);
}
// end
//tree diameter
ll dia = 0, nxt;
void tdfs(int child, int parent, int dis=0) {
 if (dis>dia)
  dia = dis, nxt = child;
 for (auto c:adj[child]) {
  if (c==parent)
   continue;
  tdfs(c,child,dis+1);
 }
}
// Code for solving system of 2 linear equations
pair<double,double> fun(const tuple<double,double,double
    >&p, const tuple<double,double,double>&q) {
 auto &[a1,b1,c1] = p;
 auto &[a2,b2,c2] = q;
 double det = a1*b2-a2*b1;
 if (det==0)
  return {inf,inf};
 double dx = c1*b2-c2*b1, dy = c2*a1-c1*a2;
 return {dx/det,dy/det};
}
// code end
```

```
//euler tour
struct segtree {
 ll NEUTRAL_ELEMENT = 0;
 ll size;
 vector<ll>sums;
 ll single (ll val) {
  return val;
 }
 ll merge (ll a, ll b) {
  return a+b;
 }
 void init (ll n) {
  size = 1;
  while (size<n)
   size *= 2;
  sums.assign(2*size,NEUTRAL_ELEMENT);
 }
 void build (vector<ll>&v, ll x, ll lx, ll rx) {
  if (rx-lx==1) {
   if (lx<(ll)v.size())
    sums[x] = single(v[lx]);
   return;
  }
  ll mid = (lx+rx)/2;
  build(v,2*x+1,lx,mid);
  build(v,2*x+2,mid,rx);
  sums[x] = merge(sums[2*x+1],sums[2*x+2]);
 }
 void build (vector<ll>&v) {
  build(v,0,0,size);
 }
 void set (ll i, ll val, ll x, ll lx, ll rx) {
  if (rx-lx==1) {
   sums[x] = single(val);
   return;
  }
  ll mid = (lx+rx)/2;
  if (i<mid)
   set(i,val,2*x+1,lx,mid);
  else
   set(i,val,2*x+2,mid,rx);
  sums[x] = merge(sums[2*x+1],sums[2*x+2]);
 }
 void set(ll i, ll val) {
  set(i,val,0,0,size);
 }
 ll query(ll l, ll r, ll x, ll lx, ll rx) {
  if (lx>=r || rx<=l)
   return NEUTRAL_ELEMENT;
  if (lx>=l && rx<=r)
   return sums[x];
  ll mid = (lx+rx)/2;
  ll s1 = query(l,r,2*x+1,lx,mid);
  ll s2 = query(l,r,2*x+2,mid,rx);
  return merge(s1,s2);
 }
 ll query(ll l, ll r) {
  return query(l,r,0,0,size);
 }
};
vector<ll> adj[N], subtree(N, 1), pos(N), val, v(N);
void dfs(ll child, ll par) {
 val.pb(v[child]);
 pos[child] = (ll)val.size() - 1;
 for (auto &c : adj[child]) {
  if (c == par)
   continue;
  dfs(c, child);
  subtree[child] += subtree[c];
 }
}
void pre() {
}
void init (ll n) {
}
void solve() {
 ll n, q;
 cin >> n >> q;
 for (ll i = 0; i < n; ++i)
  cin >> v[i];
 for (ll i = 0; i < n-1; ++i) {
  ll a, b;
  cin >> a >> b;
  a--, b--;
  adj[a].pb(b);
  adj[b].pb(a);
 }
 dfs(0, -1);
 segtree st;
 st.init(n);
 st.build(val);
 while (q--) {
  ll type;
  cin >> type;
  if (type == 1) {
   ll s, x;
   cin >> s >> x;
   s--;
   st.set(pos[s], x);
  }
  else {
   ll s;
   cin >> s;
   s--;
   cout << st.query(pos[s], pos[s] + subtree[s]) << "\n";
  }
 }
}
// KMP
void kmp(string &s) {
 for (int i = 1; i < s.size(); ++i) {
  int j = pi[i - 1];
  while (s[i] != s[j] && j > 0)
   j = pi[j - 1];
  if (s[i] == s[j])
   j++;
  pi[i] = j;
 }
}
void compute_automaton(string s, vector<vector<int>>& aut
    ) {
 s += '#';
 int n = s.size();
 vector<int> pi = prefix_function(s);
 aut.assign(n, vector<int>(26));
 for (int i = 0; i < n; i++) {
  for (int c = 0; c < 26; c++) {
   int j = i;
   while (j > 0 && 'a' + c != s[j])
    j = pi[j-1];
   if ('a' + c == s[j])
    j++;
   aut[i][c] = j;
  }
 }
}
//Trie
struct Node {
 Node *links[26] = {nullptr};
 int cntEndWith = 0, cntPrefix = 0;
 bool containsKey(char ch) {
  return (links[ch - 'a'] != nullptr);
 }
 void put(char ch, Node* node) {
  links[ch - 'a'] = node;
 }
 Node* get(char ch) {
  return links[ch - 'a'];
 }
 void increaseEnd() {
```

```
  cntEndWith++;
 }
 void increasePrefix() {
  cntPrefix++;
 }
 void deleteEnd() {
  cntEndWith--;
 }
 void reducePrefix() {
  cntPrefix--;
 }
 int getEnd() {
  return cntEndWith;
 }
 int getPrefix() {
  return cntPrefix;
 }
};
class Trie {
private:
 Node* root;
 void clear(Node* node) {
  if (!node) return;
  for (int i = 0; i < 26; i++) {
   if (node->links[i]) {
    clear(node->links[i]);
   }
  }
  delete node;
 }
public:
 Trie() {
  root = new Node();
 }
 ~Trie() {
  clear(root);
 }
 void insert(const string &word) {
  Node* node = root;
  for (auto &c : word) {
   if (!node->containsKey(c))
    node->put(c, new Node());
   node = node->get(c);
   node->increasePrefix();
  }
  node->increaseEnd();
 }
 int countWordsEqualTo(const string &word) {
  Node* node = root;
  for (auto &c : word) {
   if (!node->containsKey(c))
    return 0;
   node = node->get(c);
  }
  return node->getEnd();
 }
 int countWordsStartingWith(const string &prefix) {
  Node* node = root;
  for (auto &c : prefix) {
   if (!node->containsKey(c))
    return 0;
   node = node->get(c);
  }
  return node->getPrefix();
 }
 void erase(const string &word) {
  if (countWordsEqualTo(word) == 0)
   return;
  Node* node = root;
  for (auto &c : word) {
   node = node->get(c);
   node->reducePrefix();
  }
  node->deleteEnd();
 }
};
//end
```

6 Math

6.1 Linear Sieve

```
const int N = 1e7;
vector<int> primes;
int spf[N + 5], phi[N + 5], NOD[N + 5], cnt[N + 5], POW[N
    + 5];
bool prime[N + 5];
int SOD[N + 5];
void init() {
 fill(prime + 2, prime + N + 1, 1);
 SOD[1] = NOD[1] = phi[1] = spf[1] = 1;
 for (LL i = 2; i <= N; i++) {
  if (prime[i]) {
   primes.push_back(i), spf[i] = i;
   phi[i] = i - 1;
   NOD[i] = 2, cnt[i] = 1;
   SOD[i] = i + 1, POW[i] = i;
  }
  for (auto p : primes) {
   if (p * i > N or p > spf[i]) break;
   prime[p * i] = false, spf[p * i] = p;
   if (i % p == 0) {
    phi[p * i] = p * phi[i];
    NOD[p * i] = NOD[i] / (cnt[i] + 1) * (cnt[i] + 2),
        cnt[p * i] = cnt[i] + 1;
    SOD[p * i] = SOD[i] / SOD[POW[i]] * (SOD[POW[i]] + p
        * POW[i]),
        POW[p * i] = p * POW[i];
    break;
   } else {
    phi[p * i] = phi[p] * phi[i];
    NOD[p * i] = NOD[p] * NOD[i], cnt[p * i] = 1;
    SOD[p * i] = SOD[p] * SOD[i], POW[p * i] = p;
   }
  }
 }
}
```

6.2 Pollard Rho

```
LL mul(LL a, LL b, LL mod) {
 return (__int128) a * b % mod;
 // LL ans = a * b - mod * (LL) (1.L / mod * a * b);
 // return ans + mod * (ans < 0) - mod * (ans >= (LL) mod
     );
}
LL bigmod(LL num, LL pow, LL mod) {
 LL ans = 1;
 for (; pow > 0; pow >>= 1, num = mul(num, num, mod)){
  if (pow & 1) ans = mul(ans, num, mod);
 }
 return ans;
}
bool is_prime(LL n) {
 if (n < 2 or n % 6 % 4 != 1) return (n | 1) == 3;
 LL a[] = {2, 325, 9375, 28178, 450775, 9780504,
     1795265022};
 LL s = __builtin_ctzll(n - 1), d = n >> s;
 for (LL x : a) {
  LL p = bigmod(x % n, d, n), i = s;
  for (; p != 1 and p != n - 1 and x % n and i--; p = mul
      (p, p, n));
  if (p != n - 1 and i != s) return false;
 }
 return true;
}
LL get_factor(LL n) {
 auto f = [&](LL x) { return mul(x, x, n) + 1; };
 LL x = 0, y = 0, t = 0, prod = 2, i = 2, q;
 for (; t++ % 40 or gcd(prod, n) == 1; x = f(x), y = f(f(
     y))) {
  (x == y) ? x = i++, y = f(x) : 0;
```

```
  prod = (q = mul(prod, max(x, y) - min(x, y), n)) ? q :
      prod;
 }
 return gcd(prod, n);
}
map<LL, int> factorize(LL n) {
 map<LL, int> res;
 if (n < 2) return res;
 LL small_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
      31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83,
      89, 97};
 for (LL p : small_primes) {
  for (; n % p == 0; n /= p) res[p]++;
 }
 auto _factor = [&](LL n, auto &_factor) {
  if (n == 1) return;
  if (is_prime(n))
   res[n]++;
  else {
   LL x = get_factor(n);
   _factor(x, _factor);
   _factor(n / x, _factor);
  }
 };
 _factor(n, _factor);
 return res;
}

vector<LL> get_all_factors(LL n) {
  if (n <= 0) return {};

  map<LL, int> prime_factors = factorize(n);

  vector<LL> factors = {1};

  for (auto const& [p, count] : prime_factors) {
    int current_size = factors.size();
    LL p_pow = 1;

    for (int i = 0; i < count; i++) {
      p_pow *= p;
      for (int j = 0; j < current_size; j++) {
        factors.push_back(factors[j] * p_pow);
      }
    }
  }
  sort(factors.begin(), factors.end());
  return factors;
}
```

6.3 Chinese Remainder Theorem

```
// given a, b will find solutions for
// ax + by = 1
tuple<LL, LL, LL> EGCD(LL a, LL b) {
 if (b == 0)
  return {1, 0, a};
 else {
  auto [x, y, g] = EGCD(b, a % b);
  return {y, x - a / b * y, g};
 }
}
// given modulo equations, will apply CRT
PLL CRT(vector<PLL> &v) {
 LL V = 0, M = 1;
 for (auto &[v, m] : v) { // value % mod
  auto [x, y, g] = EGCD(M, m);
  if ((v - V) % g != 0) return {-1, 0};
  V += x * (v - V) / g % (m / g) * M, M *= m / g;
  V = (V % M + M) % M;
 }
 return make_pair(V, M);
}
```

6.4 Mobius Function

```
const int N = 1e6 + 5;
int mob[N];
void mobius() {
 memset(mob, -1, sizeof mob);
 mob[1] = 1;
 for (int i = 2; i < N; i++)
  if (mob[i]) {
   for (int j = i + i; j < N; j += i) mob[j] -= mob[i];
  }
}
```

7 String

7.1 Double hash

```
// define +, -, * for (PLL, LL) and (PLL, PLL), % for (
    PLL, PLL);
PLL base(1949313259, 1997293877);
PLL mod(2091573227, 2117566807);

PLL power(PLL a, LL p) {
 PLL ans = PLL(1, 1);
 for(; p; p >>= 1, a = a * a % mod) {
   if(p & 1) ans = ans * a % mod;
 }
 return ans;
}
PLL inverse(PLL a) { return power(a, (mod.ff - 1) * (mod.
    ss - 1) - 1); }
PLL inv_base = inverse(base);
PLL val;
vector<PLL> P;

void hash_init(int n) {
 P.resize(n + 1);
 P[0] = PLL(1, 1);
 for (int i = 1; i <= n; i++) P[i] = (P[i - 1] * base) %
     mod;
}
PLL append(PLL cur, char c) { return (cur * base + c) %
    mod; }
/// prepends c to string with size k
PLL prepend(PLL cur, int k, char c) { return (P[k] * c +
    cur) % mod; }
/// replaces the i-th (0-indexed) character from right
    from a to b;
PLL replace(PLL cur, int i, char a, char b) {
 cur = (cur + P[i] * (b - a)) % mod;
 return (cur + mod) % mod;
}
/// Erases c from the back of the string
PLL pop_back(PLL hash, char c) {
 return (((hash - c) * inv_base) % mod + mod) % mod;
}
/// Erases c from front of the string with size len
PLL pop_front(PLL hash, int len, char c) {
 return ((hash - P[len - 1] * c) % mod + mod) % mod;
}
/// concatenates two strings where length of the right is
     k
PLL concat(PLL left, PLL right, int k) { return (left * P
    [k] + right) % mod; }
/// Calculates hash of string with size len repeated cnt
    times
/// This is O(log n). For O(1), pre-calculate inverses
PLL repeat(PLL hash, int len, LL cnt) {
 PLL mul = (P[len * cnt] - 1) * inverse(P[len] - 1);
 mul = (mul % mod + mod) % mod;
 PLL ret = (hash * mul) % mod;
 if (P[len].ff == 1) ret.ff = hash.ff * cnt;
 if (P[len].ss == 1) ret.ss = hash.ss * cnt;
 return ret;
}
LL get(PLL hash) { return ((hash.ff << 32) ^ hash.ss); }
struct hashlist {
 int len;
```

```
vector<PLL> H, R;
hashlist() {}
hashlist(string& s) {
 len = (int)s.size();
 hash_init(len);
 H.resize(len + 1, PLL(0, 0)), R.resize(len + 2, PLL(0,
     0));
 for (int i = 1; i <= len; i++) H[i] = append(H[i - 1],
     s[i - 1]);
 for (int i = len; i >= 1; i--) R[i] = append(R[i + 1],
     s[i - 1]);
}
/// 1-indexed
PLL range_hash(int l, int r) {
 return ((H[r] - H[l - 1] * P[r - l + 1]) % mod + mod) %
      mod;
}
PLL reverse_hash(int l, int r) {
 return ((R[l] - R[r + 1] * P[r - l + 1]) % mod + mod) %
      mod;
}
PLL concat_range_hash(int l1, int r1, int l2, int r2) {
 return concat(range_hash(l1, r1), range_hash(l2, r2),
     r2 - l2 + 1);
}
PLL concat_reverse_hash(int l1, int r1, int l2, int r2)
    {
 return concat(reverse_hash(l2, r2), reverse_hash(l1, r1
     ), r1 - l1 + 1);
}
};
```

7.2 Suffix Array

```
vector<VI> c;
VI sort_cyclic_shifts(const string &s) {
 int n = s.size();
 const int alphabet = 256;
 VI p(n), cnt(alphabet, 0);

 c.clear();
 c.emplace_back();
 c[0].resize(n);

 for (int i = 0; i < n; i++) cnt[s[i]]++;
 for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
 for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;

 c[0][p[0]] = 0;
 int classes = 1;
 for (int i = 1; i < n; i++) {
  if (s[p[i]] != s[p[i - 1]]) classes++;
  c[0][p[i]] = classes - 1;
 }

 VI pn(n), cn(n);
 cnt.resize(n);
 for (int h = 0; (1 << h) < n; h++) {
  for (int i = 0; i < n; i++) {
   pn[i] = p[i] - (1 << h);
   if (pn[i] < 0) pn[i] += n;
  }
  fill(cnt.begin(), cnt.end(), 0);
  /// radix sort
  for (int i = 0; i < n; i++) cnt[c[h][pn[i]]]++;
  for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
  for (int i = n - 1; i >= 0; i--) p[--cnt[c[h][pn[i]]]]
      = pn[i];

  cn[p[0]] = 0;
  classes = 1;

  for (int i = 1; i < n; i++) {
   PII cur = {c[h][p[i]], c[h][(p[i] + (1 << h)) % n]};
   PII prev = {c[h][p[i - 1]], c[h][(p[i - 1] + (1 << h))
        % n]};
   if (cur != prev) ++classes;
   cn[p[i]] = classes - 1;
  }
  c.push_back(cn);
 }
 return p;
}
VI suffix_array_construction(string s) {
 s += "!";
 VI sorted_shifts = sort_cyclic_shifts(s);
 sorted_shifts.erase(sorted_shifts.begin());
 return sorted_shifts;
}
/// LCP between the ith and jth (i != j) suffix of the
    STRING
int suffixLCP(int i, int j) {
 assert(i != j);
 int log_n = c.size() - 1;

 int ans = 0;
 for (int k = log_n; k >= 0; k--) {
  if (c[k][i] == c[k][j]) {
   ans += 1 << k;
   i += 1 << k;
   j += 1 << k;
  }
 }
 return ans;
}

VI lcp_construction(const string &s, const VI &sa) {
 int n = s.size();
 VI rank(n, 0);
 VI lcp(n - 1, 0);

 for (int i = 0; i < n; i++) rank[sa[i]] = i;

 for (int i = 0, k = 0; i < n; i++, k -= (k != 0)) {
  if (rank[i] == n - 1) {
   k = 0;
   continue;
  }
  int j = sa[rank[i] + 1];
  while (i + k < n && j + k < n && s[i + k] == s[j + k])
      k++;
  lcp[rank[i]] = k;
 }
 return lcp;
}
```

7.3 Z Algo

```
vector<int> calcz(string s) {
 int n = s.size();
 vector<int> z(n);
 int l = 0, r = 0;
 for (int i = 1; i < n; i++) {
  if (i > r) {
   l = r = i;
   while (r < n && s[r] == s[r - l]) r++;
   z[i] = r - l, r--;
  } else {
   int k = i - l;
   if (z[k] < r - i + 1) z[i] = z[k];
   else {
    l = i;
    while (r < n && s[r] == s[r - l]) r++;
    z[i] = r - l, r--;
   }
  }
 }
 return z;
}
```